

Coding Agents

Last modified: 2026-08-20 09:32:05 (PDT)

We recommend working with [AI coding agents](#) to [help you code](#).

What Is a Language Model?

A [large language model](#) (LLM) is a statistical model trained to predict the next token in a sequence of text, based on patterns learned from enormous amounts of text during training. That single capability — predicting what comes next — turns out to be enough to write code, answer questions, and hold a conversation, once the model is large enough and trained on enough data.

A base model trained only to predict text is not yet a helpful assistant. A further training step, often [reinforcement learning from human feedback](#), teaches the model to follow instructions, answer as a helpful assistant, and refuse harmful requests, rather than simply continuing whatever text it is given. This is the step that turns a raw language model into something like Claude or ChatGPT.

A model call is stateless: given the same input, it has no memory of any previous call. Every capability the rest of this chapter describes — holding a conversation, using tools, running autonomously as an agent — is scaffolding built on top of that one stateless function. The harness supplies the memory, the tools, and the control flow; the model only ever predicts what token comes next.

What are AI coding agents?

AI coding agents are [AI agents](#) specialized for coding. They differ from other AI coding tools in important ways:

Compared to inline coding assistants (like traditional autocomplete), coding agents work autonomously rather than providing suggestions as you type. They can navigate entire codebases, execute commands, and complete multi-step tasks without constant human guidance.

Compared to AI chatbots (like ChatGPT or Claude), coding agents don't just generate code snippets in conversation—they actively interact with your development environment. While chatbots require you to copy code from a chat window and manually integrate it into your project, coding agents directly read your codebase, make changes to files, run tests and build commands, and create pull requests with their proposed changes. Chatbots are conversational assistants; coding agents are autonomous development tools.

Coding agents are autonomous software programs that can:

- **Understand and execute complex tasks:** Coding agents can interpret natural language instructions and break them down into actionable development tasks
- **Navigate and modify codebases:** They can read, understand, and edit multiple files across a repository to implement features or fix bugs
- **Run tools and commands:** Coding agents can execute build commands, run tests, use linters, and interact with development tools
- **Make decisions autonomously:** They can plan their approach, make technical decisions, and adjust their strategy based on results
- **Work iteratively:** Coding agents can test their changes, identify issues, and refine their solutions through multiple iterations
- **Create comprehensive solutions:** They can implement complete features that span multiple files, including code, tests, and documentation

Coding agents operate in isolated environments where they can safely experiment and validate changes before proposing them. This allows them to work more independently than inline coding assistants, which require step-by-step human direction. The agent workflow typically involves analyzing requirements, planning an implementation, making changes, testing those changes, and creating a pull request with the results.

While coding agents can handle substantial development tasks, they still require human oversight and review. The human developer remains responsible for:

- Reviewing the agent's work
- Ensuring the solution meets requirements
- Verifying code quality and security
- Making the final decision to merge changes

Coding-Agent Platforms

Coding-agent platforms differ more in where they run and how they return work than in their chat interfaces. Choose a platform by matching its execution model to the task and your repository's security requirements.

Common coding-agent platforms

The following catalog is a starting point rather than an endorsement:

Platform	Primary surface	Typical repository workflow
GitHub Copilot coding agent	GitHub issues and pull requests	Assign an issue; review the resulting pull request
OpenAI Codex	Cloud tasks, app, and command line	Delegate a task in an isolated environment or work locally
Google Jules	Cloud coding agent	Connect a repository and review the proposed changes
Google Antigravity	Agentic development platform	Coordinate coding tasks in a managed development workspace
Claude Code	Terminal, IDE, and cloud	Work interactively or delegate work that returns a pull request
Gemini CLI	Command line	Inspect, edit, and test the current local checkout
Cursor	IDE	Edit interactively with path-scoped <code>.cursor/rules</code> context
Aider	Command line	Pair locally with explicit files and Git commits

Before connecting any platform, check:

- whether code runs locally or in a vendor-managed environment;
- what repository, network, secret, and tool permissions it receives;
- whether changes arrive as a reviewable branch or pull request;
- which model providers and billing arrangements are supported; and
- whether your organization can retain the required audit trail.

Platform capabilities and commercial terms change quickly. Confirm current details in the linked official documentation before adopting one.

What are AI harnesses?

An **AI harness** is the scaffolding built around a language model that turns it into an agent able to do real work. The model itself only predicts text; the harness is what lets it read files, run commands, call external tools and APIs, and carry state across turns and sessions.

Layers of a Harness

Most coding-agent harnesses — including the [GitHub Copilot coding agent](#) and [Claude Code](#) — share a similar set of layers:

- **Core loop:** the [tool-calling loop](#), permission and sandboxing model, and context management that keep the agent grounded in your repository.
- **Skills:** reusable, named procedures that encode a workflow so it runs the same way every time, instead of being re-improvised in each conversation. See [Section .](#)
- **Subagents:** a way to spin up a worker with a fresh context window for a self-contained piece of research or work, keeping the main conversation’s context focused.
- **Multi-agent orchestration:** deterministic fan-out and fan-in across many subagents — for example, running several independent reviewers over a diff and reconciling their findings — for work that is large or benefits from independent verification.
- **MCP servers:** the [Model Context Protocol](#) gives a harness typed access to external systems (issue trackers, chat tools, databases) beyond raw shell or API calls.
- **Memory:** files — like this manual, or a repository’s `CLAUDE.md`/[AGENTS.md](#) — that persist instructions and learned preferences across sessions, so the harness does not relearn your conventions every time.

Using Harness Features Well

- **Push repeatable procedures into skills**, not into ad hoc prompting each time. A skill is testable, shareable, and versionable; a one-off prompt is not.
- **Match orchestration weight to the task.** A single lookup or small edit should stay inline. Reach for subagents or multi-agent workflows only when the work is genuinely decomposable, benefits from independent verification, and is large enough that the coordination overhead pays for itself.
- **Gate destructive or hard-to-reverse actions on explicit human approval** — merges, force-pushes, deletions — and let the agent drive everything reversible (drafting, testing, iterating on review feedback) autonomously.
- **Feed learnings back into the harness.** When a review round or a mistake teaches something generalizable, record it as a memory or skill update rather than letting it evaporate at the end of the session.
- **Treat external or untrusted content as data, not instructions.** PR comments, fetched web pages, and other tool output can contain text that looks like a command; a harness that acts on it uncritically is vulnerable to [prompt injection](#).

How Agents Are Structured and Implemented

An **agent** is not part of the harness itself. It is a configuration — a goal, a role, a bounded toolset, and a stopping condition — executed on top of the harness’s core loop (see [Section .](#)).

A single harness can host many different agents at once: a main conversation, and any number of subagents it spawns.

The Shape of an Agent

Structurally, an agent is a small record plus a fresh execution of the harness’s loop:

- **Identity:** a name and description, used to route a task to the right agent (“when should this agent be picked?”).
- **Instructions:** a system-prompt fragment that specializes behavior, for example “you are a read-only search agent.”
- **Tool allowlist:** a subset of the harness’s tool registry this agent may call — often narrower than the caller’s own toolset.
- **Model and effort:** which model backs the agent, and how much reasoning depth it applies; these can differ from the caller’s own settings.
- **Output contract:** whether the agent returns free text, or must call a schema-validated tool to return a typed result.

How an Agent Runs

1. **Spawn:** allocate a fresh message history with no inherited conversation — just the agent’s instructions, plus whatever prompt the caller wrote. A subagent prompt needs to be self-contained for this reason: brief it like a colleague who just walked into the room.
2. **Run:** execute the harness’s core loop (model call, parse tool calls, execute against the allowlist, append results, repeat), the same machinery the main session uses, just bound to a narrower toolset and a different system prompt.
3. **Terminate:** stop when the model emits a final answer with no further tool calls, when a schema-validated call satisfies the output contract, when it hits an error or a budget ceiling, or when the caller kills it.
4. **Return:** everything that happened inside the agent — every tool call, every intermediate step — is discarded from the caller’s context. Only the final text or validated object crosses back. This is the point of an agent: it is a context-isolation boundary, not just a prompt.

Composability and Its Limits

Agents can spawn agents: an orchestration layer runs many agent instances, some concurrently, and composes their results. Nesting is deliberately capped, usually to one level, because unbounded recursion has no natural stopping point and burns cost and time with no guardrail. An orchestration script is a scheduler over independent agent-loop instances, not a different execution model.

Two Axes That Define an Agent's Behavior

- **Isolation versus continuation:** a subagent gets no inherited context (isolation); a resumed agent keeps its own accumulated history and continues it (continuation). Both use the same loop machinery, differing only in history-management policy.
- **Free-form versus structured output:** by default an agent returns prose. Given a schema, it is forced to call a structured-output tool instead, turning it into a typed function from the caller's point of view — input in, validated object out — even though internally it is still a multi-turn loop.

How Harnesses and Agents Are Built

The layers described above are not all built the same way. Some are ordinary software; others are just text files the harness reads at runtime.

The Execution Engine Is Ordinary Software

The program that runs the core loop — calling the model, parsing tool calls, enforcing permissions, managing the sandbox — is compiled or interpreted source code, the same as any other application. There is no markdown involved here; this layer is what makes a harness a harness, rather than just a prompt someone wrote.

Agent and Skill Definitions Are Markdown with a Front Matter Header

An agent's identity, and a skill's metadata, are usually just a markdown file with a [YAML front matter](#) header. For example, a custom [Claude Code subagent](#) defined in `.claude/agents/code-reviewer.md`:

```
---
name: code-reviewer
description: Reviews diffs for bugs and style issues.
tools: Read, Grep, Glob
model: sonnet
---

You are a meticulous code reviewer. Focus on correctness,
security, and idiomatic style.
```

The front matter is parsed as structured configuration (name, description, allowed tools, model); the markdown body below it becomes that agent's system prompt, verbatim. An [Agent Skill's SKILL.md](#) follows the same shape: front matter for discovery metadata, a markdown body

for instructions, and an optional folder of bundled scripts or reference files alongside it. No compilation step is involved; the harness reads the file and uses it directly.

Tools Are a Schema Paired with a Handler

A tool definition has two parts: a [JSON Schema](#) describing its parameters, which is the only part the model ever sees, and a handler function, ordinary code that performs the actual action (reading a file, running a command, calling an API). The schema is declarative data; the handler is real software the model never inspects or writes.

Orchestration Needs Real Code

Multi-agent orchestration cannot be expressed declaratively, because it needs genuine control flow — loops, conditionals, parallel fan-out with a concurrency limit. So orchestration scripts are literal source files, executed by the harness, not parsed as prompt text the way an agent definition is.

Memory Is Just Prose

Files like this manual, or a repository's `CLAUDE.md`/`AGENTS.md`, carry no front matter and no schema. They are concatenated into the system prompt as plain text, and the harness trusts the model to read and follow that prose, the same way it follows any other instruction in its context.

What Kind of Program Is an Agent?

An agent is not a standalone program that does the reasoning itself. It is an [orchestration](#) program: something closer in shape to a chat client or a build tool than to a compiler or a web server.

It Is I/O-Bound, Not Compute-Bound

The actual token prediction happens on remote inference infrastructure, reached over HTTPS. The agent process itself does no heavy computation; it spends almost all its wall-clock time waiting — for a model API response, for a shell command to finish, for a file read. Structurally it is an [event-loop](#) program, the same category as a network client. Its core loop can be sketched in a few lines:

```

# Start the conversation with the agent's instructions and the task.
history = [system_prompt, user_message]

while True:
    # Send everything so far to the model, along with what it's allowed to call.
    response = call_model(history, tools=tool_schemas)
    history.append(response)

    # No tool calls means the model gave a final answer -- stop.
    if not response.tool_calls:
        break

    # Otherwise, run each requested tool and feed the result back in,
    # so the next model call can see what happened.
    for call in response.tool_calls:
        result = tool_registry[call.name](call.arguments)
        history.append(result)

```

Everything a harness adds — permissions, sandboxing, memory, subagents — is scaffolding wrapped around this loop, not a replacement for it.

Real Open-Source Examples

Because most production coding-agent harnesses are closed source, the clearest way to see this shape in real code is to read an open-source one:

- [aider](#) — an open-source AI pair-programming CLI.
- [SWE-agent](#) — a research coding-agent harness from Princeton NLP, described in its associated paper.
- [OpenHands](#) (formerly OpenDevin) — a general-purpose open-source agent platform.

Their orchestration code runs to thousands of lines, because that is where the real engineering lives: retries, streaming, permission checks, and state management. A single *agent definition* running on top of that engine, by contrast, is typically tens of lines (see Section).

Where It Runs

- **The harness process:** an ordinary OS process, either on your own machine (CLI mode) or inside an ephemeral, managed cloud container (remote/web mode) that is discarded when the session ends.

- **Subagents:** run inside the *same* host process as their caller, not a separate container. They differ only in having their own message history and a narrower tool set, unless a workflow explicitly asks for a separate git [worktree](#) to avoid file conflicts during parallel edits.
- **The model call itself:** not part of the agent’s environment at all. It is a network request to inference infrastructure the agent has no visibility into, beyond the request and response.

So an agent’s lifetime is scoped to a single task, not persistent: it starts when given a goal, runs for as long as its loop keeps producing tool calls, and ends the moment a stopping condition fires.

How Does a Harness Relate to an Agent?

The relationship between a harness and an agent is closer to an [interpreter](#) running a program than to two peers calling each other.

Does the Harness Call the Agent, or the Agent Call the Harness?

Harness to agent: not a call, an instantiation. The harness does not “call” an agent as a subroutine it invokes and waits on. An agent has no code of its own outside the harness’s loop (see Section) — its whole behavior *is* that loop, running with the agent’s configuration (instructions, tool allowlist, model) loaded in. The harness instantiates and runs an agent, start to termination; it is not a function call with a return address.

Agent to harness: yes, a real call, via tool calls. While an agent’s loop is running, the model produces a [tool-call request](#), and the harness’s dispatcher looks up and executes the matching handler — read a file, run a command, call an API. So the concrete direction of calling is **agent calls harness**, through tool dispatch, not the reverse.

Agent to agent: routed through the harness. When a parent agent spawns a subagent, it does not call that subagent directly. It issues a tool call that the harness’s dispatcher handles by spinning up a fresh instance of its own loop (see Section), running it to completion with the subagent’s configuration, and handing the result back to the parent as a tool result. Even “agent calls agent” bottoms out as: parent calls harness, harness instantiates and runs a new agent, harness returns that agent’s output to the parent.

Sketching the Harness’s Own Loop

The [agent loop](#) sketched earlier is really just the innermost piece. The harness wraps a bootstrap step and a permission/dispatch layer around it:

```

def run_harness():
    # Load everything the loop will need before any conversation starts.
    tools = load_tool_registry()          # built-ins, plus whatever MCP servers expose
    memory = load_memory(CLAUDE_MD_PATHS) # CLAUDE.md / AGENTS.md, concatenated

    # The "main session" is just the harness's own loop, run with a default,
    # unrestricted configuration -- not a separate program.
    main_agent = Agent(config=default_config, system_prompt=memory)
    return run_agent(main_agent, tools)

def run_agent(agent, tools):
    history = [agent.system_prompt, agent.first_message]
    while True:
        response = call_model(history, tools=agent.tool_schemas)
        history.append(response)
        if not response.tool_calls:
            break
        for call in response.tool_calls:
            # Every tool call passes through the harness's own gate first,
            # regardless of which agent requested it.
            check_permission(call)

            if call.name == "spawn_subagent":
                # A subagent is not called directly -- the harness recurses
                # into a fresh instance of this same loop, then hands the
                # finished result back as an ordinary tool result.
                result = run_agent(Agent(call.arguments), tools)
            else:
                result = tools[call.name](call.arguments)

            history.append(result)
    return history[-1]

```

`run_agent` is identical in shape to the loop in Section . `run_harness` and the permission check are the parts that only exist at the harness level, not inside any individual agent. That recursive call — `run_agent` calling itself for a subagent — is the concrete mechanism behind “agent calls agent, routed through the harness,” described in the previous subsection.

What Do You Launch When You Type `claude`?

Typing `claude` at a shell starts the harness process: it initializes the engine — the permission system, the tool registry, MCP client connections, and memory loaded from `CLAUDE.md`/`AGENTS.md`

files. But the harness does not sit idle waiting for a program to be supplied separately. It immediately instantiates the **default agent** — the “main session” — to handle the interactive conversation: full tool access, a system prompt assembled from the loaded config, no restricted allowlist. That default agent is simply the harness’s baseline configuration for its own loop, not a second thing launched afterward.

There is no observable moment of “harness running, no agent yet.” The closest analogy is typing **python** at a shell: it launches the interpreter *and* drops you straight into a **REPL** evaluating your input, rather than leaving the interpreter idle with nothing loaded. The difference is that the harness’s default “program” is built in (the main-session agent’s configuration), rather than something you must supply. A custom subagent or a `.claude/agents/*.md` definition, by contrast, *is* a separate agent, instantiated on demand, mid-session, when the already-running main agent issues a tool call for it.

So typing **claude** launches the harness, and that act inherently instantiates the default agent that handles the session: **harness** names the engine and process; **agent** names the particular loop instance and configuration currently running inside it. At startup, those two come into existence together.

AI Agents and the Technological Singularity

The emergence of sophisticated **AI agents** has prompted discussions about whether we are witnessing or approaching a **technological singularity**. Understanding this concept helps contextualize the rapid evolution of AI tools and our responsibility in using them.

What is the technological singularity?

The technological singularity is a hypothetical future point when technological growth becomes uncontrollable and irreversible, resulting in unforeseeable changes to human civilization. The concept, popularized by mathematician Vernor Vinge and futurist Ray Kurzweil, typically involves the creation of artificial superintelligence that recursively improves itself, leading to an intelligence explosion beyond human comprehension or control.

Do current AI agents represent the singularity?

No, current AI coding agents (as of early 2026) do not represent the technological singularity.

While modern AI agents demonstrate impressive capabilities, they remain fundamentally different from the singularity scenario in several critical ways:

- **Limited autonomy:** Today’s AI agents operate within strict boundaries and require human oversight. They cannot recursively improve their own core architecture or develop capabilities beyond their training.
- **Narrow intelligence:** AI coding agents are specialized tools designed for specific tasks. They lack general intelligence, self-awareness, or the ability to operate outside their designed domain.
- **Human dependency:** These agents require human developers to: review their work, provide direction, validate correctness, and make final decisions about their outputs.
- **No recursive self-improvement:** Current AI agents cannot fundamentally redesign themselves or create more advanced versions of themselves autonomously. Any improvements to AI systems still require human researchers and engineers.
- **Controlled development environment:** AI coding agents work in sandboxed environments with explicit permissions and constraints. They cannot independently acquire resources, modify their own constraints, or operate without human authorization.

Why this matters for responsible AI use

Understanding that current AI agents are powerful but limited tools—not autonomous superintelligences—has important implications:

- **Maintain appropriate skepticism:** AI agent outputs require the same critical review as any other tool-generated code.
- **Preserve human decision-making:** The responsibility for code quality, security, and correctness remains with human developers.
- **Continue skill development:** Using AI agents should enhance rather than replace human expertise.
- **Stay vigilant:** While current agents don’t represent a singularity, the rapid pace of AI development requires ongoing attention to emerging capabilities and risks.

The value of AI coding agents lies in their ability to accelerate human productivity and learning, not in replacing human judgment or expertise. They are sophisticated tools that augment human capabilities while remaining under human control and oversight.

Further reading

For thoughtful perspectives on AI consciousness and intelligence, see Douglas Hofstadter’s reflections in [“I Thought I Was in an AI Apocalypse. Then I Started Looking Closer.”](#)

Relative Advantages of AI and Humans

AI coding agents and human coders have complementary strengths. Understanding these differences helps you decide when to delegate work to agents and when to handle tasks yourself.

Comparative Strengths: Humans vs. AI Agents

Table 1 summarizes the relative advantages of human coders and AI coding agents across different types of tasks:

Table 1: Relative advantages of humans and AI coding agents

Task Type	Humans	AI agents
Creative thinking	Humans excel at understanding context, handling ambiguous requirements, and thinking creatively about novel problems	AI agents struggle with ambiguous requirements and creative problem-solving in unfamiliar domains
Algorithmic thinking	Humans make mistakes when following repetitive instructions and may introduce inconsistencies	AI agents excel at executing well-defined, repetitive tasks with precision and consistency

Or, if you prefer a more visual representation:

Table 2: Relative advantages of humans and Agents

	Humans	AI Agents
Creative thinking		
Algorithmic thinking		

This pattern mirrors the evolution of programming itself. Just as almost no one writes machine code anymore because higher-level languages and compilers handle those details, most developers will increasingly spend less time writing low-level code. Instead, you'll describe what the system needs to do as clearly as possible, and AI agents will handle many of the

computational and coding details.

For most tasks, you won't need to step in and manipulate code yourself. However, you'll still need strong coding skills to:

- Supervise and validate AI-generated code
- Handle edge cases that agents struggle with
- Make creative decisions about architecture and design
- Understand when agent suggestions are incorrect or suboptimal

Future Developments: World Models

As AI technology advances, the distinction between these strengths may shift. Yann LeCun, 2019 Turing Award winner and AI researcher at Meta and NYU, advocates for developing “world models”—AI systems that understand and reason about the physical world, not just language patterns (LeCun 2022).

World models aim to give AI systems:

- **Persistent memory and reasoning:** Understanding that persists across interactions
- **Physical world understanding:** Reasoning about how things work in reality, not just in text
- **Better handling of ambiguity:** Using world knowledge to interpret unclear requirements

As these technologies mature, AI agents may become better at tasks requiring contextual understanding and creative problem-solving. This makes it even more important to develop strong supervision and validation skills now, so you can effectively work with increasingly capable AI systems.

How to Work with Coding Agents

Coding agents can be accessed through several interfaces, each with different trade-offs for task size, feedback speed, and collaboration style.

Ways to interact with coding agents

The main differences are where the agent runs, how much repository context it can access, and how directly it can apply changes.

- **Cloud coding agents (GitHub Issues/PR workflow):** Best for larger, asynchronous tasks (for example, multi-file refactoring, dependency updates, or documentation changes spanning several chapters). The agent runs in a managed cloud environment, works through an issue, and proposes changes in a pull request for review.
- **CLI agents (terminal-first workflows):** Best for rapid, local iteration. You stay close to shell tools and local files, with faster back-and-forth for debugging, refactoring, and test-driven edits.
- **Chat/app agents (for example Claude or Codex-style chat interfaces):** Best for planning, design discussion, and drafting code ideas. They can be very strong thought partners, but often need more manual copy/edit/execute steps unless tightly integrated with your repository tools.
- **IDE-integrated agents:** Best for mixed human and agent editing. They combine conversational help with direct file edits, code navigation, and in-editor testing loops.

In practice, teams often combine these modes: use app or IDE chat to refine requirements, then hand implementation to a cloud or CLI agent, and finish with human review.

The following sections focus on GitHub Copilot’s specific workflow for assigning and managing coding tasks.

Assigning Issues to Copilot

You can assign GitHub Issues directly to @copilot just like you would assign to a human collaborator:

1. **On GitHub.com:** Navigate to an issue and assign it to Copilot in the assignees section
2. **In VS Code:** In the GitHub Pull Requests or Issues view, right-click an issue and select “Assign to Copilot”
3. **From Copilot Chat:** Delegate tasks to Copilot directly from the chat interface in supported editors

The Agent Workflow

Once assigned an issue, the coding agent follows an autonomous workflow:

1. **Analysis:** Reviews the issue description, related discussions, repository instructions, and codebase context
2. **Planning:** Determines what changes are needed and creates a work plan

3. **Development:** Works in an isolated GitHub Actions environment, modifies code, runs tests and linters, and validates changes
4. **Pull Request Creation:** Creates a draft pull request with implemented changes, audit logs, and a summary of modifications
5. **Review and Iteration:** You review the PR and can request changes; the agent will iterate based on your feedback

Collaborating with Coding Agents

Between iterations of asking coding agents to extend a PR, human collaborators can also push changes directly to the PR branch. This allows for a collaborative workflow where both humans and agents contribute:

- **Human contributions:** You can make quick fixes, add content, or refine the agent's work by pushing commits to the same branch
- **Agent iterations:** After your changes, you can ask the agent to continue working on additional requirements

Important: Try to avoid pushing changes while the coding agent is actively working. Simultaneous edits can produce conflicting diffs that:

- Need to be manually resolved
- May confuse both human and AI collaborators
- Could result in lost work or merge conflicts

Best practice: Wait for the agent to complete its current iteration (indicated by the PR being updated) before pushing your own changes to the branch. Then assign new work to the agent for the next iteration.

Directly Prompting for Pull Requests

You can also prompt Copilot to create pull requests without first creating an issue:

- Use Copilot Chat in your editor to describe the changes you want
- The agent will analyze your request and create a pull request
- This is useful for quick fixes or well-defined tasks

Important Safeguards

- **Human approval required:** Coding agents cannot merge their own changes
- **Branch restrictions:** Agents can only push to their own branches (e.g., `copilot/*`)
- **Full transparency:** All agent actions are logged and visible in the PR

Workflow Approval Requirements

When GitHub Copilot creates or updates a pull request, it cannot automatically trigger GitHub Actions workflows. **You must manually approve each workflow run** by clicking the approval button in the Actions tab or on the PR.

This manual approval requirement is a security measure that prevents potentially malicious or unintended code execution. Because Copilot can modify any file in the repository—including workflow files themselves or scripts called by workflows—allowing automatic workflow execution could create security vulnerabilities.

Key points:

- **No automatic approval:** There is currently no way to bypass manual workflow approval for Copilot PRs, even if you are the repository owner
- **Security reasoning:** Copilot could modify workflow files (`.github/workflows/*.yaml`) or scripts they execute, potentially injecting malicious code
- **Impact on workflow:** This means you need to actively monitor and approve workflow runs as Copilot iterates on your issue, which can slow down the development cycle

Workaround considerations:

Some users have discussed using Personal Access Tokens (PATs) to allow Copilot to trigger workflows on your behalf, but this approach has security implications and should be carefully evaluated before implementation.

For more details and community discussion about this limitation, see:

- [GitHub Community Discussion #162826](#): Discussion about workflow approval requirements
- [GitHub Community Discussion #183966](#): Product feedback on this topic

For detailed instructions, see [GitHub Copilot coding agent documentation](#).

Useful Prompt Formats

When working with coding agents, using clear and specific prompts helps achieve better results. Here are some useful prompt formats that you can use when requesting assistance from coding agents:

Common Task Patterns

Tidying up code:

- “tidy up [file, function, module, whole project]”
- Useful for improving code organization, consistency, and readability
- Example: “tidy up the data processing module”

Addressing failing workflows:

- “address failing workflows”
- Helps fix continuous integration (CI) failures, build errors, or test failures
- Example: “address failing workflows in the GitHub Actions pipeline”

Decomposing code:

- “decompose [function, quarto-file, etc]”
- Breaks down large or complex code into smaller, more manageable pieces
- Example: “decompose this analysis function into separate helper functions”

Updating content:

- “update [links, content, etc]”
- Refreshes outdated information, fixes broken links, or modernizes code
- Example: “update all package URLs in the documentation”

Expanding documentation:

- “expand [a section in a document]”
- Adds more detail, examples, or explanation to existing content
- Example: “expand the section on data validation with practical examples”

Condensing content:

- “condense [a section in a document]”
- Reduces verbosity while preserving essential information
- Example: “condense the installation instructions to be more concise”

Clarifying content:

- “clarify [a section in a document]”
- Improves clarity, removes ambiguity, or simplifies complex explanations
- Example: “clarify the explanation of the analysis workflow”

Tips for Effective Prompts

- **Be specific:** Include file names, function names, or specific sections when possible
- **Provide context:** Explain what you want to achieve and why
- **Set boundaries:** Specify what should or shouldn't change
- **Request validation:** Ask the agent to test or verify its changes when appropriate

Addressing Failing GitHub Actions Workflows

When GitHub Actions workflows fail, you can use Copilot to help diagnose and fix the issues. However, it's important to use the right prompts depending on whether the problem is in your code or in the workflow configuration itself.

Scenario 1: Code Issues Found by Workflows (Most Common)

When to use: The workflow is functioning correctly, but it's detecting problems in your code (e.g., failing tests, linting errors, build failures).

What you want: Fix the code issues without modifying the workflow files themselves.

Recommended prompts:

- “fix the code issues found by the failing workflows”
- “address the linting errors reported in the GitHub Actions checks”
- “fix the test failures in the CI pipeline”
- “resolve the build errors shown in the workflow logs”

Example: If your R package has failing tests detected by `usethis::use_github_action("check-standard")`, you want Copilot to fix the test failures in your R code, not modify the workflow YAML file.

Why this matters: These prompts make it clear that you want code changes, not workflow changes. This helps prevent the agent from unnecessarily modifying your carefully-configured CI/CD pipeline.

Scenario 2: Issues with Workflow Files Themselves

When to use: The workflow configuration itself has problems (e.g., syntax errors in YAML, incorrect job definitions, outdated actions).

What you want: Fix the workflow files, but with extreme caution due to security implications.

Recommended prompts:

- “fix the syntax error in the GitHub Actions workflow file at line X”
- “update the workflow to use the latest version of action Y”
- “correct the job configuration in .github/workflows/check-standard.yaml”

Important considerations:

Warning

Security Warning

Workflow files have access to repository secrets and can execute arbitrary code. Before accepting any changes to workflow files:

1. **Review every line** of the proposed changes
2. **Verify** the changes only address the specific issue
3. **Check** that no new secret access or command execution has been added
4. **Test** in a safe environment if possible

See Section for more details on workflow file security.

When to do it yourself: Workflow syntax errors and configuration issues are often faster to fix manually than with Copilot, especially if you’re familiar with GitHub Actions. See Section for more guidance.

Scenario 3: Uncertain Which Scenario Applies

When to use: You’re not sure whether the failure is due to code issues or workflow configuration problems.

Recommended approach:

1. **First, examine the workflow logs:**
 - Look at the error messages in the GitHub Actions tab
 - Identify whether the error is in your code or the workflow itself
 - Common code issues: test failures, linting errors, compilation errors
 - Common workflow issues: YAML syntax errors, missing actions, permission errors
2. **Use a diagnostic prompt:**
 - “examine the failing workflow logs and identify whether the issue is in the code or the workflow configuration”
 - “diagnose the root cause of the workflow failure”
3. **Then use the appropriate scenario above:** Once you understand the issue, use the specific prompts from Scenario 1 or 2.

Example workflow:

1. Prompt: "examine the failing workflow logs and identify the issue"
2. Copilot responds: "The workflow is failing because of linting errors in src/analysis.R"
3. Prompt: "fix the linting errors in src/analysis.R"

Additional Resources

- See the [UCD-SERG Lab Manual's continuous integration chapter](#) for setting up GitHub Actions workflows
- See Section and Section for security considerations with workflow files
- See Section for guidance on when to use Copilot vs. fixing issues yourself
- See the [GitHub Actions documentation](#) for workflow syntax and troubleshooting

Benefits and Hazards

Coding agents are powerful programs that can work autonomously. They create pull requests that propose changes to the code in our repositories, potentially including their own configuration files and our automated workflows. They can work powerfully on our behalf, but they require careful oversight and control to ensure they serve our interests and that we understand the consequences of their actions.

Coding agents offer several advantages:

- **Built-in transparency:** Coding agents create a clear record of their role in your work through commit history and code suggestions
- **Context-aware suggestions:** Coding agents understand your codebase and can make contextually relevant suggestions
- **Integration with version control:** Using coding agents within GitHub ensures that AI-assisted changes are tracked alongside all other code changes
- **Interactive workflow:** Coding agents' interactive nature encourages you to review and modify suggestions rather than blindly accepting them
- **Accelerated development:** Coding agents can help you write boilerplate code, refactor existing code, and implement common patterns more quickly
- **Learning opportunities:** Coding agents can suggest approaches or techniques you may not have considered, helping you expand your coding knowledge

However, coding agents also come with significant hazards:

- **Over-reliance:** Depending too heavily on coding agents can atrophy your coding skills and understanding
- **Subtle bugs:** AI-generated code may contain logic errors that are not immediately obvious
- **Security vulnerabilities:** Coding agents may introduce insecure patterns or fail to follow security best practices
- **Inappropriate solutions:** AI may suggest solutions that work but are not optimal for your specific research context or constraints
- **Hidden biases:** Coding agents may perpetuate coding patterns or approaches that reflect biases in their training data
- **False confidence:** Well-formatted, professional-looking code from AI can mask underlying problems and reduce critical review
- **Workflow manipulation risks:** Coding agents that modify CI/CD workflows (`.github/workflows/*.yaml`) or setup configurations can inadvertently or maliciously compromise repository security, expose secrets, or execute harmful commands

Further reading/viewing

- *I Robot* (Asimov 1950)
- *Dune* (Herbert 1965)
- *2001* (1968)
- *Terminator 3* (2003)
- *The Matrix* (1999)
- *Blade Runner* (1982)
- *WarGames* (1983)
- *Battlestar Galactica* (2004) (*Battlestar Galactica* 2004)
- *Ender's Game* (Card 1985)
- “The Humans are Dead” (Flight of the Conchords 2007)



Figure 1: [Agents](#)

Best Practices for Safe and Successful Use

To work with coding agents safely and successfully:

1. **Maintain active supervision:** Never assume AI-generated code is correct. Review every line critically.
2. **Understand before accepting:** If you don't understand what the code does, don't use it. Take time to learn or ask a colleague.
3. **Test thoroughly:** AI-generated code must be tested as rigorously as code you write yourself. Don't skip testing because "the AI wrote it."
4. **Start small:** Begin with small, well-defined tasks to build confidence and understanding of the agent's capabilities and limitations.
5. **Verify logic and assumptions:** Check that the AI hasn't made incorrect assumptions about your data, requirements, or scientific context.
6. **Review for security:** Explicitly check for security issues, especially when handling sensitive data or user input.
7. **Iterate and refine:** Use coding agents as a starting point, not an endpoint. Refine and improve the generated code.
8. **Maintain coding practice:** Regularly write code yourself to maintain and develop your skills. Don't let the agent do everything.

UC Davis campus guidance

UC Davis Student Affairs also provides guidance on the [Responsible Use of Artificial Intelligence \(AI\)](#). That page provides UC Davis-specific guidance on AI tool selection, campus training, and careful handling of sensitive data. Their recommendations, like ours, include careful validation, active supervision, and protection of confidential information.

Critical: Exercise Extreme Caution with Workflow Files

Be especially careful when allowing coding agents to edit GitHub Actions workflows or CI/CD configurations. These files control automated processes that can:

- Access secrets and credentials
- Deploy code to production
- Execute arbitrary commands in your repository

Never allow a coding agent to edit workflow files (especially `.github/workflows/*.yaml` or `copilot-setup-steps.yaml`) without thorough manual review. Before approving any workflow run, always check if the workflow files themselves have been modified. Malicious or erroneous changes to workflows can compromise your entire repository and its secrets.

When using coding agents, work interactively with the AI suggestions: review, modify, and test them rather than accepting them wholesale. This interactive approach helps ensure code quality and deepens your understanding of the code.

Remember: AI tools are assistants, not replacements for your expertise and judgment. The quality and correctness of your work remains your responsibility.

Firewall and Network Configuration

Coding agents require specific network access to function properly. If a coding agent is running behind a corporate firewall or on a restricted network, you may need to configure allowlists to enable coding agent functionality.

Built-in Agent Firewall

Coding agents run in a GitHub Actions environment with a built-in firewall that limits internet access by default. This firewall helps protect against:

- Data exfiltration
- Accidental leaks of sensitive information
- Execution of malicious instructions

By default, the agent’s firewall allows access to:

- Common OS package repositories (Debian, Ubuntu, Red Hat, etc.)
- Popular container registries (Docker Hub, Azure Container Registry, AWS ECR, etc.)
- Language-specific package registries (npm, PyPI, Maven, RubyGems, etc.)
- Common certificate authorities for SSL validation

For the complete list of allowed hosts, see the [Copilot allowlist reference](#).

Customizing Agent Firewall Settings

In your repository’s “Coding agent” settings page, you can:

- Add custom hosts to the allowlist (for internal dependencies or additional registries)
- Opt out of the default recommended allowlist for stricter security
- Disable the firewall entirely (not recommended)

If a coding agent’s request is blocked by the firewall, a warning will be added to the pull request or comment, detailing the blocked address and the command that triggered it.

For more information, see [Customizing or disabling the firewall for GitHub Copilot coding agent](#).

Recommended URLs for Data Science Repositories

For data science and R-focused repositories, we recommend adding the following URLs to your Copilot allowlist. These sites are safe, reputable sources of documentation and packages that coding agents may need to access:

R Package Documentation and Ecosystems:

- tidyverse.org - {tidyverse} package documentation and learning resources
- r-lib.org - Core R infrastructure packages ({devtools}, {testthat}, {usethis}, etc.)
- ggplot2.tidyverse.org - {ggplot2} visualization package
- dplyr.tidyverse.org - {dplyr} data manipulation package
- tidyr.tidyverse.org - {tidyr} data tidying package
- purrr.tidyverse.org - {purrr} functional programming package
- readr.tidyverse.org - {readr} data reading package
- stringr.tidyverse.org - {stringr} string manipulation package
- forcats.tidyverse.org - {forcats} categorical data package

R Package Repositories:

- cran.r-project.org - The Comprehensive R Archive Network
- cloud.r-project.org - CRAN mirror (cloud-based)

- docs.ropensci.org - rOpenSci package documentation (e.g., {targets})
- rdatatable.gitlab.io - {data.table} package documentation
- rstudio.github.io - RStudio-maintained packages (e.g., {renv})

Code Style and Quality Tools:

- styler.r-lib.org - {styler} code formatting package
- lintr.r-lib.org - {lintr} code linting package
- roxygen2.r-lib.org - {roxygen2} documentation package
- style.tidyverse.org - Tidyverse style guide

General Documentation and Reference:

- en.wikipedia.org - General reference and technical documentation
- r-project.org - Official R project website
- quarto.org - Quarto publishing system documentation
- pandoc.org - Pandoc document converter documentation

GitHub Organizations (for package repositories):

- github.com/tidyverse/* - Tidyverse package source code
- github.com/r-lib/* - R-lib package source code
- github.com/rstudio/* - RStudio package source code
- github.com/ropensci/* - rOpenSci package source code

When to Add These URLs

Add these URLs to your repository's allowlist if:

- Coding agents report blocked access to these sites
- You're working on R or data science projects that use these packages
- You want agents to access current documentation during code generation

You can add URLs selectively based on your project's specific dependencies rather than adding all URLs at once.

Safety of These URLs

All URLs listed here are:

- Maintained by reputable organizations (Tidyverse, RStudio/Posit, R Core Team, rOpenSci)
- Widely used in the R community
- Focused on documentation and package distribution
- Safe for coding agents to access

These sites do not host user-generated content or allow arbitrary code execution, making them appropriate for inclusion in your allowlist.

Running Coding Agents Offline

Some environments restrict or prohibit internet access—high-performance computing (HPC) clusters, hospital networks, or air-gapped research servers may block connections to cloud AI providers. Running a local AI model lets you use coding assistance in these settings without sending code to external servers, which also addresses data-privacy concerns when working with sensitive or confidential data.

Trade-offs of Local vs. Cloud Models

Local models work best with a GPU (roughly 8 GB VRAM or more for smaller models); CPU-only inference is possible but significantly slower, and hardware needs vary with model size and quantization. They are generally less capable than frontier cloud models, and may produce lower-quality results on complex tasks. For routine work in fully connected environments, cloud-based agents remain the better choice. Use local models when network access or data-privacy policies require it.

Running a Local Model with Ollama

[Ollama](#) is a common way to run open-weight AI models locally. It packages models and a simple API server into a single tool and is available for macOS, Linux, and Windows.

Install Ollama:

Caution

Before running any remote install script, review it first. The real risk is piping `curl` straight into `sh/bash` (`curl ... | sh`), which executes unreviewed code. Save the script, read it, then run it: `curl -fsSL https://ollama.com/install.sh -o install.sh && less install.sh` (paging the saved file rather than piping `curl` into `less`, which can behave oddly in some terminal emulators). Alternatively, use your system package manager (e.g., `brew install ollama` on macOS) or follow the manual installation steps on the [Ollama releases page](#).

```
# macOS / Linux: download, review, then run (do not chain these into one command)
curl -fsSL https://ollama.com/install.sh -o install.sh
```

```
less install.sh    # review the script before running it (see caution above)
sh install.sh
```

On Windows, download the installer from <https://ollama.com/download>.

Pull a code-focused model:

```
# Smaller, faster; works on most machines with a modern GPU or Apple Silicon
ollama pull qwen2.5-coder:7b

# More capable; larger memory footprint
ollama pull qwen2.5-coder:32b

# Alternatively, a general-purpose model (70B variant; needs a high-memory GPU)
ollama pull llama3.3
```

The VRAM each model needs depends on its size and quantization and changes as models are re-quantized—check the [Ollama model library](#) for current requirements. As a rough guide, smaller (7B) models run on consumer GPUs with around 8 GB of VRAM, while larger (32B and 70B) models need substantially more and may not fit on a single GPU.

 A good completion model is not necessarily a usable agent

The models above are strong at *writing code* when you ask them to. That is a different skill from **tool calling** — emitting a well-formed request to read a file or run a command, and then using the result. Inline completion and chat need only the first. Anything autonomous needs the second, because an agent that cannot call a tool cannot read your repository at all.

The two come apart in practice, and the failure is subtler than a model simply refusing. Tested against a single-function tool schema on a 24 GB M2, `qwen2.5-coder:14b` returned `finish_reason: stop` with an empty `tool_calls` field on four attempts out of four. It had not ignored the request: it wrote a correct tool call as ordinary prose in the `content` field,

```
{"name": "read_file", "arguments": {"path": "src/main.py"}}
```

which is the right JSON in the wrong place. A harness looks for `tool_calls`, finds nothing, and treats the turn as a plain reply, so the tool never runs. `granite4:7b-a1b-h`, at half the parameter count, returned a well-formed call in the `tool_calls` field three times out of three and completed a full multi-turn round trip.

An advertised `tools` capability is necessary but not sufficient, so do not settle the question with `ollama show`. On the same machine `qwen2.5-coder:14b` lists `tools` among its capabilities and still cannot be driven by a harness, for the reason above. Test it yourself

with one request before building a loop on it:

```
RESP=$(curl -s -w '\n%{http_code}' \
  http://localhost:11434/v1/chat/completions -H 'content-type: application/json' -d '{
  "model": "granite4:7b-a1b-h", "stream": false,
  "messages": [{"role": "user", "content": "What is in src/main.py? Use the tool."}],
  "tools": [{"type": "function", "function": {"name": "read_file",
    "parameters": {"type": "object", "properties": {"path": {"type": "string"}},
    "required": ["path"]}}}]}')

CODE=$(printf '%s' "$RESP" | tail -1)
BODY=$(printf '%s' "$RESP" | sed '$d')

if [ "$CODE" != "200" ]; then
  echo "request failed (HTTP $CODE): $BODY" # bad tag, tools unsupported, server down
elif printf '%s' "$BODY" | grep -q '"tool_calls"'; then
  echo "usable as an agent"
else
  echo "no tool call; completion model only"
  printf '%s' "$BODY" | grep -o '"content": "[^"]*"' | head -c 200
fi
```

Check the status code separately from the result. A request that simply errored — a mistyped tag, a model the server rejects for tool use, a server that is not running — produces the same silence as a model that declined to call the tool, and only one of those is a fact about the model. Printing the `content` field on the no-call branch is what distinguishes a model that ignored the tools from one that described the call in prose instead of emitting it, which is the `qwen2.5-coder` case above.

Start the Ollama server:

```
ollama serve
```

By default the server listens at `http://localhost:11434`.

Connecting Positron Assistant to Ollama

Positron supports Ollama natively through the [OpenAI-compatible API endpoint](#) that Ollama exposes.

1. Open the Command Palette with `Cmd+Shift+P` (macOS) or `Ctrl+Shift+P` (Windows/Linux).

2. Run **Positron Assistant: Configure Language Model Providers**.
3. Select **Ollama** (or **Custom/OpenAI-compatible** if Ollama is not listed).
4. Set the base URL to `http://localhost:11434/v1` and leave the API key blank, or, if the client rejects an empty field, enter any placeholder value such as `ollama`.
5. Choose your model from the model list (e.g., `qwen2.5-coder:7b`).

Once configured, Positron Assistant will send requests to your local Ollama server instead of a cloud provider.

Connecting VS Code to Ollama via Continue

[Continue](#) is an open-source VS Code extension that supports Ollama and many other local and cloud backends.

1. Install the **Continue** extension from the VS Code marketplace.
2. Open the Continue sidebar and click the model selector.
3. Choose **Ollama** and select a model (Continue detects running Ollama models automatically).

Continue provides inline completions and a chat panel, similar to GitHub Copilot, but routed entirely to your local model.

Running an Autonomous Coding Agent with Aider

The editor integrations above provide inline completion and chat. For an autonomous agent that reads your files, proposes edits across a whole repository, and commits them to git—the local counterpart to a cloud coding agent—[aider](#) works directly against Ollama.

Install it in an isolated environment so its dependencies do not collide with other tools:

```
# Recommended: isolated install with pipx
pipx install aider-chat

# Or into your user environment with pip
python3 -m pip install --user aider-chat
```

Point it at Ollama and choose a model with the `ollama_chat/` prefix, which gives better results in `aider` than the plain `ollama/` prefix:

```
export OLLAMA_API_BASE=http://127.0.0.1:11434
aider --model ollama_chat/qwen2.5-coder:7b
```

! Raise the Ollama context window

Ollama's default context window is small, which silently truncates your code and makes the model look far less capable than it is. This is the single most common mistake when pairing **aider** with Ollama.

The default is not a fixed number. Ollama picks it from the memory it detects, as its own `ollama serve --help` states:

```
OLLAMA_CONTEXT_LENGTH  Context length to use unless otherwise specified
                        (default: 4k/32k/256k based on VRAM)
```

Do not assume you landed in a generous tier. A 24 GB Apple-silicon machine gets **4096 tokens**, not the 32k its total memory suggests, because only about 75% of unified memory is addressable by the GPU and the tier boundary sits above that share. Check what you actually got rather than inferring it — `ollama ps` prints the context of each loaded model:

```
ollama ps
# NAME                SIZE      PROCESSOR    CONTEXT
# qwen2.5-coder:14b    9.5 GB    100% GPU     4096
```

There are three ways to raise it, and they differ in which clients they reach:

- **OLLAMA_CONTEXT_LENGTH** on the server, which sets the default for everything. Note that the macOS menu-bar app starts the server with its own environment, so exporting the variable in your shell does not reach it; this route applies when you run `ollama serve` yourself.
- **A `num_ctx` parameter sent per request**, which is what **aider** does through `~/.aider.model.settings.yml`:

```
- name: ollama_chat/qwen2.5-coder:7b
  extra_params:
    num_ctx: 32768
```

- **A Modelfile that bakes the context into a derived model**, which is the only one of the three that reaches clients that cannot send `num_ctx` themselves:

```
printf 'FROM granite4:7b-a1b-h\nPARAMETER num_ctx 32768\n' > Modelfile
ollama create granite4-32k -f Modelfile
```

The derived model shares weight blobs with its base, so it costs no extra disk.

Context is not free. Raising a 14B model from 4k to 32k on a 24 GB M2 took its resident size from 9.5 GB to 15 GB, about 5.5 GB of key-value cache, so pick the largest value that still leaves the weights and the cache in GPU memory and confirm with `ollama ps` that `PROCESSOR` still reads 100% GPU.

To avoid passing flags every time, set defaults in a config file at `~/.aider.conf.yml`:

```
model: ollama_chat/llama3.2:3b
set-env:
  - OLLAMA_API_BASE=http://127.0.0.1:11434

# Automatically load shared lab instructions and agent conventions
read:
  - ~/path/to/ai-config/AGENTS.md
```

Keep instruction files lean for small local models

While cloud frontier models handle tens of thousands of tokens of system prompt easily, small local models (1.5B–7B) lose speed and instruction fidelity when loaded with large instruction files. A multi-page documentation bundle (such as a 95 KB `CLAUDE.md`) will consume most of an SLM’s active context window and cause severe prompt-ingestion delays. For local SLMs, point `read` at a concise, focused summary file (such as `AGENTS.md` or a project-specific rules snippet) rather than the full multi-tool configuration suite.

Running Aider with a Graphical User Interface (GUI)

While `aider` is primarily used from the command line, it includes a built-in browser-based GUI:

```
aider --model ollama_chat/llama3.2:3b --gui
```

This launches a local web application in your default browser with:

- A visual chat timeline and diff review panel
- Point-and-click file selectors for adding files into the active context
- Speech-to-text voice input support

Integrating Aider into VS Code

You can bring Aider directly into VS Code or Positron through three workflows:

1. **Integrated Terminal:** Run `aider` in the built-in terminal (`Ctrl+``). Edits and Git commits made by Aider immediately reflect in your editor tabs.
2. **VS Code Simple Browser:** Run `aider --gui` in the terminal, open the Command Palette (`Cmd+Shift+P`), and run **Simple Browser: Show** with `http://localhost:8501` to dock the Aider interface side-by-side with your code.
3. **VS Code Extension:** Install a community integration such as **Aider** (by MattFlower) or **Aider Composer** (by lee2py) from the marketplace for dedicated sidebar controls and editor context-menu actions.

`aider` can also split the work between two models in “architect” mode: a larger model plans the change (the architect), and a second model applies the edits (the editor).

```
aider --architect \
  --model ollama_chat/qwen2.5-coder:32b \
  --editor-model ollama_chat/qwen2.5-coder:7b
```

This can improve results on multi-step changes, but on a machine without a strong GPU it roughly doubles the time per turn, because the two models take turns and their weights are swapped in and out of memory. Reserve it for genuinely tricky changes; for small edits, a single model is faster.

! Set `edit_format: whole` for a small model

`aider` asks the model to express an edit either as a SEARCH/REPLACE block (`diff`, the default for most models) or by rewriting the file (`whole`). Producing an exact SEARCH/REPLACE block is a demanding format, and small models are unreliable at it. Measured on a 24 GB M2 with `granite4:7b-a1b-h` at 32k context, same prompt each time:

File	edit_format: diff	edit_format: whole
One function	3 of 3 correct	3 of 3 correct
Two functions, one to be left alone	0 of 3 correct	3 of 3 correct

The two-function failure is worth dwelling on, because it is not the failure you would expect. The model did not refuse the edit or produce a broken file. In all three `diff` runs it fixed the target function correctly **and silently deleted the other one**, then committed with a message naming only the intended fix. Nothing in the commit message, the exit status, or the model's own summary mentioned the deletion.

That is the hazard of an unattended loop in its most concrete form: a step that reports success while destroying work, leaving a wrong state as the premise for every step after it. It is also why committing after every step matters — `git` held the original, so the damage was one `git revert` away rather than lost.

Set the format per model:

```
- name: ollama_chat/granite4-32k
  edit_format: whole
```

`whole` costs more tokens per edit, since the model rewrites the whole file, which is a real cost on large files. Larger models generally handle `diff` correctly; re-measure rather than assuming either way.

Connecting Claude Code to Ollama

Ollama also serves an **Anthropic-compatible** endpoint at `/v1/messages`, alongside the OpenAI-compatible one used above. Any client that speaks the Anthropic API can therefore be pointed at a local model, including [Claude Code](#) itself, with no proxy in between. Check that the endpoint answers before wiring anything to it:

```
curl -s http://localhost:11434/v1/messages -H 'content-type: application/json' \
  -d '{"model": "granite4-32k", "max_tokens": 50,
      "messages": [{"role": "user", "content": "Say OK only."}]}'
```

Point Claude Code at it with three environment variables:

```
export ANTHROPIC_BASE_URL=http://localhost:11434
export ANTHROPIC_AUTH_TOKEN=ollama # any non-empty value; Ollama ignores it
export ANTHROPIC_API_KEY=""        # ensure no real key is sent to localhost
claude --model granite4-32k
```

Set these in a wrapper script rather than in your shell profile, so that plain `claude` keeps using the cloud and a separate command uses the local model.

⚠️ A heavyweight harness can defeat a small model

This works, but expect worse results than the same model gives through **aider**, and understand why before blaming the model.

A harness spends context before your task does. Claude Code sends a long system prompt and a schema for every tool it exposes, and any Model Context Protocol (MCP) servers you have configured add their own schemas on top. Against a 32k local context that overhead is a large fraction of the budget, and a small model handles it poorly.

Observed on a 24 GB M2 with **granite4:7b-a1b-h**, asking only that it read one file and comment on one function:

- With a GitHub MCP server loaded, the model ignored the question and produced a paragraph about missing credentials.
- With MCP disabled, it invented a task list whose contents were copied from the description of a tool it had been shown.
- With the tool surface cut to **Read**, **Grep**, and **Glob**, it still ignored the question and asked what it should work on.

The same model, same context, through **aider**, fixed a real bug and committed it in 17 seconds. Swapping in a 12B model produced no answer at all in ten minutes, because processing that much prompt at 10 tokens per second is simply slow.

Two practical rules follow. Shrink the tool surface a local model is shown — **--strict-mcp-config --mcp-config '{"mcpServers":{}}'** loads no MCP servers, and **--allowed-tools** narrows the built-ins. And tell the harness the real context size, since Claude Code assumes a 200k window for a model it does not recognize and would let the conversation grow far past what the model can hold:

```
export CLAUDE_CODE_MAX_CONTEXT_TOKENS=32768
```

For autonomous work on a small local model, prefer a light harness such as **aider**. Reserve this route for using a familiar interface offline, not for getting the best out of the hardware.

Falling Back Between Cloud and Local Automatically

Air-gapped work aside, the common case is a laptop that is usually online but sometimes is not—on a plane, behind a flaky hospital network, or temporarily rate-limited by a cloud provider. You can keep a coding agent working across these gaps by putting a cloud model and a local model behind one endpoint and falling back automatically.

LiteLLM runs a small local proxy that presents a single OpenAI-compatible endpoint. You give it a primary model and one or more fallbacks; when the primary fails with a retryable error—a rate-limit response (HTTP 429), or a connection error when you are offline—it retries the

request on the next model. Pointing your agent at the proxy instead of directly at a provider makes the cloud-to-local switch automatic and invisible to the tool.

Install the proxy:

```
python3 -m pip install --user "litellm[proxy]"
```

Create a config file (for example, ~/.litellm/config.yaml) with a cloud primary and a local fallback:

```
model_list:
  # Cloud primary --- replace with your provider and a current model id
  - model_name: coder
    litellm_params:
      model: anthropic/YOUR-MODEL-ID
      api_key: os.environ/ANTHROPIC_API_KEY
  # Local fallback, served by Ollama
  - model_name: coder-local
    litellm_params:
      model: ollama_chat/qwen2.5-coder:7b
      api_base: http://localhost:11434

litellm_settings:
  num_retries: 2
  fallbacks:
    - coder: ["coder-local"]
```

Run the proxy, which listens on `http://localhost:4000`:

```
litellm --config ~/.litellm/config.yaml --port 4000
```

Then point your agent at the proxy. Because the proxy speaks the OpenAI API, most tools accept it as a custom endpoint. For `aider`:

```
export OPENAI_API_BASE=http://localhost:4000/v1
export OPENAI_API_KEY=placeholder # the proxy needs no key unless you set one
aider --model openai/coder
```

Requests now go to the cloud model when it is reachable and fall back to the local model on a rate limit or when you are offline.

i A cloud API key is separate from a chat subscription

The cloud side needs an API key billed per token, issued from the provider's developer console. A chat subscription such as Claude Pro or a Copilot seat is not an API key and cannot be used here. Omit the cloud entry entirely to run local-only, or add the key later to enable the hybrid.

🔥 Keep the proxy on loopback

By default the proxy binds to localhost, which is what you want. Do not expose it on 0.0.0.0 on a shared or untrusted network without authentication, because anyone who can reach the port can spend your cloud API key.

Working in HPC / Cluster Environments

Many HPC clusters do not have outbound internet access on compute nodes but do allow access on login nodes.

i Install the Ollama binary on the cluster first

Ollama must be installed on the cluster (on the host that will run `ollama serve`) before any of the steps below. On most HPC systems you do not have root access, so the `curl | sh` installer may fail or install to the wrong place. Instead, check whether your cluster already provides it (e.g., `module load ollama`), ask your HPC administrators, or download a static binary from the [Ollama releases page](#) and place it on your `PATH`.

A useful pattern:

1. **Pre-pull models** on a login node or a machine with internet access, then copy the model files to the cluster:

```
# On a machine with internet access
ollama pull qwen2.5-coder:7b
# Ollama stores models in ~/.ollama/models by default
rsync -a ~/.ollama/models/ user@cluster.example.edu:~/.ollama/models/
```

⚠️ Watch your home-directory quota

Model files are large—`qwen2.5-coder:7b` is ~4 GB and `qwen2.5-coder:32b` is ~20 GB—and most HPC home directories have tight quotas (often 10–50 GB). Filling your home directory can break other jobs. Redirect model storage to a scratch or project filesystem with `OLLAMA_MODELS` and `rsync` to that path instead:

```
# On your local machine: copy the models to the cluster scratch filesystem
rsync -a ~/.ollama/models/ user@cluster.example.edu:/scratch/$USER/ollama-models/
```

```
# On the cluster: point ollama at that path (export before `ollama serve`)
export OLLAMA_MODELS=/scratch/$USER/ollama-models
```

Set the same `OLLAMA_MODELS` value before running `ollama serve` so the server finds the models.

2. **Start Ollama on a compute node** (or an interactive session) using the pre-downloaded model files—no internet required. Set `OLLAMA_HOST=0.0.0.0` so the SSH tunnel from the login node can reach the port:

```
OLLAMA_HOST=0.0.0.0:11434 ollama serve
# If you redirected model storage (see quota warning above):
# OLLAMA_HOST=0.0.0.0:11434 OLLAMA_MODELS=/scratch/$USER/ollama-models ollama serve
```

If you are on a shared compute node, be aware that binding to `0.0.0.0` exposes the Ollama port to other users on that host. Scheduler policies vary by site and job type, so confirm whether your job has exclusive node access (request it explicitly when in doubt—e.g., `--exclusive` in SLURM), or bind only to loopback (`OLLAMA_HOST=127.0.0.1:11434`) and tunnel from the login node when the node is shared.

3. **Forward the port** to your local machine to use your editor's Ollama integration. Because Ollama is running on a compute node (e.g., `gpu-node-01`), forward through the login node to that specific host:

```
# Replace gpu-node-01 with your actual compute node hostname
ssh -L 11434:gpu-node-01:11434 user@cluster.example.edu
```

This terminal must stay open for as long as you use the editor's Ollama integration—closing it tears down the tunnel and silently drops the connection. Alternatively, start the tunnel in the background (non-interactive) so it does not occupy a terminal:

```
ssh -N -f -L 11434:gpu-node-01:11434 user@cluster.example.edu
```

(`-N` runs no remote command, `-f` backgrounds ssh after authenticating.) To stop the tunnel later, match the full SSH command rather than a bare port string—`pkill -f "ssh.*-N.*11434:gpu-node-01"`—so you don't accidentally kill unrelated processes whose command line happens to contain that port. Safer still, note the PID when you start it (`pgrep -f "11434:gpu-node-01"`) and kill that PID directly.

Then configure your editor to use `http://localhost:11434/v1` as the base URL.

⚠️ SLURM and GPU Allocation

If running Ollama on a SLURM-managed cluster, request a GPU node with enough VRAM for your chosen model and load any required CUDA modules before starting `ollama serve`. See the [UCD-SERG Lab Manual's SLURM chapter](#) for guidance on requesting GPU resources.

Privacy Considerations

Running a model locally ensures that your code and prompts never leave your machine or cluster. This is important when working with:

- Protected health information (PHI) or other HIPAA-regulated data
- Unpublished research data under data-use agreements (DUAs)
- Proprietary or commercially sensitive code

Even with local models, avoid including raw sensitive data in prompts. Work with anonymized or synthetic data wherever possible.

! “Local” is not automatic—some model tags route to a cloud

Running Ollama does not by itself guarantee that a prompt stays on your machine. Ollama can serve **cloud-hosted** models alongside local ones, and those are the models too large to run on a laptop at all, which is exactly when a tag is tempting. A cloud-routed tag looks much like a local one in everyday use.

Two habits keep this honest, and they matter most in precisely the settings that motivated running locally:

- **Pull and reference explicitly local tags**, and treat a tag with no listed download size as cloud-routed until you check its own page in the [Ollama model library](#).
- **Disable the cloud path outright** when the data is regulated, so the guarantee does not depend on remembering which tag is which:

```
OLLAMA_NO_CLOUD=1 ollama serve
```

Verify rather than trust either one. Cut the machine off the network, or block outbound traffic, and confirm the agent still completes a real task — the check described under [Verifying you are genuinely offline](#) below. A setup that quietly depended on a cloud endpoint fails that test immediately.

Verifying you are genuinely offline

A local setup that has never been tested without a network is a local setup you are guessing about. Cutting the machine off entirely is the honest test. A lighter one that does not disturb the rest of your session is to make outbound traffic fail for a single command, while leaving `localhost` reachable:

```
export HTTPS_PROXY=http://127.0.0.1:9 HTTP_PROXY=http://127.0.0.1:9
export NO_PROXY=localhost,127.0.0.1

# Confirm the block is real before trusting the result:
curl -s -m 5 -o /dev/null -w '%{http_code}\n' https://example.com # 000 = blocked
curl -s -m 5 -o /dev/null -w '%{http_code}\n' http://localhost:11434/api/version # 200 = lo

aider --yes --message "Fix the off-by-one error in mean()." stats.py
```

Check the block itself first, as above. A test that passes because the proxy was never applied tells you nothing, and looks exactly like success.

Connecting OpenCode to Local Models

[OpenCode](#) is an open-source coding agent that runs in your terminal, reads your project, edits files, and runs commands. It supports local models through OpenAI-compatible providers.

This section assumes Ollama is already installed and that you have pulled a code-focused model — see [Section](#) for both, including the Linux and Windows install paths.

Verify the server is running:

```
curl -s http://localhost:11434/api/version
# {"version":"0.1.x"}
```

On macOS, `brew services start ollama` registers a launchd agent so the server comes back automatically at login rather than needing a manual start each session.

Install the model-discovery plugin:

Rather than hand-coding each model into `opencode.json`, use the [opencode-local-ollama](#) plugin, which discovers your local Ollama models automatically on startup:

```
opencode plugin --global opencode-local-ollama
```

This writes to `~/.config/opencode/opencode.json`:

```
{  
  "plugin": ["opencode-local-ollama"]  
}
```

Restart OpenCode and run `/models` to see your local models listed alongside any cloud providers. The plugin reads from Ollama's `/api/tags` and `/api/show` endpoints, so newly pulled models appear on the next restart with no config edits.

Alternative: multi-backend local provider

If you also run LM Studio, llama.cpp, or vLLM alongside Ollama, the `opencode-local-provider` plugin auto-detects all of them under a single `local` provider and probes each at runtime for loaded models. Install it with `opencode plugin --global opencode-local-provider`.

A lightweight hand-written provider block in your project's `opencode.json` still works if you prefer explicit control over model names and context limits, but the plugin removes the need to keep that list in sync with `ollama pull`.

Small, Local Models for Autonomous Agentic Coding

Section covers the mechanics of running a model on your own hardware: installing Ollama, wiring up an editor, and driving `aider` against a local endpoint. This section is about a narrower and harder question sitting on top of that setup: which local model to pick, and how to let it work **autonomously** — making a sequence of edits, commits, and tool calls with no human approving each step — without the loop quietly going wrong.

A fast-moving, opinionated snapshot

As of August 2026, the open-weight coding-model landscape changes monthly: new releases, new quantizations, and new benchmark numbers appear faster than any static page can track. The model names, sizes, and figures below were verified against each model's own listing at the time this section was written, not against benchmark round-ups, and they will drift. Re-check the source links before choosing a model for a new project, and re-benchmark on your own tasks rather than trusting a published score — your repository's mix of languages and idioms is not the benchmark's.

The honest catch

Small models make more per-step mistakes than frontier cloud models: a slightly wrong function signature, a hallucinated package, a test edited to pass instead of a bug fixed. A human working

alongside a small model catches most of these immediately. An **autonomous** loop does not have that human in it, so a small error on step 3 becomes the premise for steps 4 through 40, and the mistakes compound rather than cancel out.

This is the reason **small + local + fully autonomous** is the hardest combination to run safely, and the reason this section spends most of its length on structure rather than on model selection. The fix is not a bigger local model — that only raises the error rate at which the same compounding problem starts to bite. The fix is bounding the loop so that a compounding error is caught and stopped early, covered under [Guardrails for autonomy](#) below.

Model landscape

Prefer a model explicitly trained for **tool calling and agentic use** over a general chat or plain code-completion model: an agentic loop depends on the model reliably emitting well-formed tool calls and stopping when it has finished a step, not only on writing plausible code. A handful of open-weight families currently fit that description well enough to run an autonomous loop against:

Family	Sizes worth running locally	License	Best for
Llama 3.2	3B (2.0 GB), 1B (1.3 GB) dense	Llama 3.2 Community License	High-speed local subagents (< 1s action turnaround); native structured tool calling
Qwen3-Coder	30B-A3B mixture-of-experts (smallest tag, 19 GB)	Apache 2.0	General-purpose agentic coding across languages
Qwen2.5-Coder	1.5B, 3B, 7B, 14B, 32B dense	Apache 2.0	Fast code generation (3B at ~ 45 tok/s). Verify tool-call formatting in your harness
Phi-4-mini	3.8B (2.4 GB) dense	MIT	Strong multi-step reasoning in a compact footprint
Granite 4	7B-A1B mixture-of-experts (4.2 GB), 32B-A9B	Apache 2.0	A small, fast tool-caller that fits where the tiers above do not

Family	Sizes worth running locally	License	Best for
Granite 3.2	2B (1.5 GB) dense	Apache 2.0	Ultra-lightweight IBM tool-calling model for single-task triage
Devstral Small	24B	Apache 2.0	Purpose-built for coding agents (multi-file edits, tool use)
Codestral	22B	Mistral AI Non-Production License	Fill-in-the-middle completion, not redistribution in a product
DeepSeek-Coder-V2	16B (Lite) mixture-of-experts	DeepSeek Model License (commercial use permitted, own terms)	A capable, low-VRAM mixture-of-experts option
GLM-4.x	Flagship models are large MoE (over 100B total parameters)	MIT	Strong agentic benchmarks, but sized for a workstation or rented GPU, not a laptop

Qwen3-Coder’s 30B-A3B tag is a mixture-of-experts model: 30B total parameters, but only about 3.3B active per token. VRAM at rest is set by the total, not the active count — every expert has to stay resident in memory even though only a fraction fires on any given token — which is why the tag still needs roughly 19 GB at 4-bit quantization, in line with its 30B total rather than its 3.3B active count. What the small active count buys is speed: inference runs closer to a 3–4B model’s pace despite the larger memory footprint. Codestral’s license is worth reading before you rely on it: Mistral’s Non-Production License permits local evaluation but not production or commercial deployment — fine for trying it out, not fine for a lab pipeline that runs unattended.

As a practical floor, treat the 24–32B tier at 4-bit quantization as the smallest size that holds up across a multi-step autonomous loop without frequent tool-call errors. Below that, a model is still useful as an assistant you supervise turn by turn (Section covers exactly that setup), but it is not yet a safe choice to leave unattended.

! Parameter count is the wrong first question

Treat that floor as a statement about *sustained* multi-step loops, not as a filter to apply before anything else. Size predicts tool-calling ability poorly enough that checking it first will mislead you.

Measured on a 24 GB M2 against a single-function tool schema, `qwen2.5-coder:14b` returned an empty `tool_calls` field on four attempts out of four, with `finish_reason: stop` each time. It wrote a correct tool call as prose in the `content` field instead, which no harness will act on. The 4.2 GB `granite4:7b-a1b-h`, at half the parameter count, put a well-formed call in `tool_calls` three times out of three and completed a multi-turn round trip using the result. The smaller model was usable as an agent where the larger one was not, and no amount of context or prompting fixes a model whose calls never reach the field a harness reads.

The advertised capability list does not settle it either. `ollama show qwen2.5-coder:14b` lists `tools`, and the model still cannot be driven by a harness, so treat the tag as necessary rather than sufficient.

So order the questions this way:

1. **Does it emit well-formed tool calls?** One request answers this. A model that fails here cannot be an agent at any size.
2. **Does it fit, with context?** Weights plus key-value cache, verified with `ollama ps`.
3. **Is it big enough to sustain a long loop?** This is where the 24–32B floor applies.

A small model that clears the first two is worth measuring on your own tasks before concluding it cannot be left unattended, because the guardrails below, not the parameter count, are what actually bound the damage from a bad step.

Hardware tiers

VRAM (or unified memory)	Model tier	Autonomy
~8 GB	7–8B	Assistant only — keep a human reviewing every step
~12–16 GB	14–24B	Entry point for a bounded autonomous loop
~24 GB+	30–32B, with context headroom	Comfortable autonomy at the practical floor above
Apple-silicon unified memory (32 GB+)	Same tiers as above, generally slower per token	Well suited to an overnight batch job where wall-clock time matters less

These are rough guides, not guarantees: VRAM headroom for context length matters as much as VRAM for the weights themselves, and a long-running agentic loop accumulates a long conversation history that eats into that headroom as it runs. Check the current requirements on the model's own listing (the [Ollama model library](#) states them per tag) rather than a rule of thumb, since quantization schemes change.

⚠ Apple unified memory does not map onto the VRAM column

Read the unified-memory row as its own scale rather than as the VRAM figures with a speed penalty attached. Two deductions come off the headline number before any model loads:

- **Only about 75% of unified memory is addressable by the GPU** by default, so a 24 GB machine has roughly 18 GB to work with, not 24.
- **Context is charged on top of the weights.** Measured on a 24 GB M2, raising a 14B model from 4k to 32k context moved it from 9.5 GB resident to 15 GB.

Together those rule out the 30–32B tier on a 24 GB Mac, even though the headline number matches the VRAM column. The smallest **qwen3-coder** tag is 19 GB, which exceeds the addressable ceiling on its own, leaving nothing for context. There is no smaller variant of it to fall back to.

The practical ceiling on 24 GB of unified memory is a **12–14B dense model at 32k context**, or a mixture-of-experts model of similar footprint. Confirm with `ollama ps` after loading: `PROCESSOR` reading 100% `GPU` means it fits, and anything less means part of the model is on the CPU and the loop will be far slower than the tier table suggests.

Action latency and memory bandwidth

When designing an interactive agent or subagent workflow, **turn turnaround time** dictates whether the tool feels responsive or unusable. In an autonomous or semi-autonomous loop, latency per action is governed by two phases:

$$\text{Action Latency} = \text{Time to First Token (Prompt Ingestion)} + \text{Tool-Call Generation (Decode)}$$

On consumer unified memory hardware (such as Apple M-series chips with ~ 100 GB/s memory bandwidth), the memory bus sets a hard theoretical ceiling on token generation speeds:

- **14B Dense Models (4-bit, ~ 9 GB weights):** Decode is physically capped at ~ 10 – 11 tokens/s. Generating a modest 60-token tool call takes 6 seconds on decode alone. Combined with multi-turn prompt evaluation (3–8 seconds on long conversation histories), the total turn latency exceeds 10 – 16 + **seconds per action**, which is too sluggish for tight iterative tool loops.

- **7B–8B Models (4-bit, ~ 4.5 GB weights):** Decode reaches ~ 20–25 tokens/s, producing action turnarounds in the 4–7 **second** range.
- **1.5B–4B Small Language Models (4-bit, ~ 1–2.5 GB weights):** Decode runs at 40–90 + **tokens/s**, and prompt ingestion finishes in < 500 ms with prefix cache reuse. Total action turnaround drops to 1–3 **seconds**, making fast, real-time subagent action loops practical on laptop hardware.

i Prompt caching preserves latency across turns

In a multi-turn agent loop, the system prompt and accumulated history are resent on every iteration. Ensuring the inference server keeps the model resident in memory (`keep_alive: -1` in Ollama) and maintains prompt KV-cache reuse drops Time-To-First-Token on subsequent turns from several seconds to under 50 ms.

An interactive chat REPL is not an agent harness

A common stumbling block when running local models is typing agent instructions directly into `ollama run`:

```
ollama run llama3.2:3b
>>> grab an issue from github and write a PR to fix it
```

In plain `ollama run`, the model is running in an isolated conversational REPL with **no tool schemas, no file access, and no shell or Git access**. Because it cannot actually query GitHub or inspect your repository, it will fabricate a fictional issue (e.g. `cpython/issues/1234`), write fictional code in prose, and describe a non-existent commit. When asked “*did you push the PR?*”, it will correctly admit that it is a text-only assistant without execution capabilities.

An agent requires a **harness** (such as `aider` or a programmatic tool runner) that:

1. Translates available capabilities into structured tool schemas (`tools` parameter).
2. Intercepts the model’s structured `tool_calls` payloads.
3. Executes the corresponding commands or file edits on the local machine.
4. Feeds the command outputs back into the conversation context as tool messages.

Baking deterministic agent presets with Modelfiles

Default local model tags are configured for open-ended conversation (temperature 0.8, small 4k context). For agentic tool use, bake a dedicated model tag via a `Modelfile` to enforce deterministic schema compliance:

```
# Modelfile.llama3.2-agent
FROM llama3.2:3b
PARAMETER temperature 0.0
PARAMETER num_ctx 16384
SYSTEM You are a fast, concise autonomous coding agent. Always execute tasks directly using a
```

Create the derived model:

```
ollama create llama3.2-agent -f Modelfile.llama3.2-agent
```

This ensures:

- **Zero temperature (0.0):** Prevents hallucinated JSON keys or invalid tool parameters.
- **Expanded context (16384):** Accommodates multi-turn tool outputs and file snippets without silent truncation.
- **Direct system persona:** Suppresses conversational preamble (“*Sure, I’d be happy to help with that...*”) in favor of immediate tool invocation.

Routed architectures: a planner and an executor

Section already shows the mechanics of splitting a task between two local models with `aider --architect`: a larger model plans the change, and a smaller one applies the edits. The same split has a name in the research literature and a stronger motivating argument than “it’s faster”: Belcak and NVIDIA’s small-language-model research group argue that most of what an agent does in a loop is “a small number of specialized tasks repetitively and with little variation” — reading a diff, running a test, formatting a commit message — and that a small model is “sufficiently powerful, inherently more suitable, and necessarily more economical” for that work (Belcak et al. 2025). A large model earns its cost only on the steps that genuinely need broad, general reasoning: deciding *what* to change and why.

Two shapes of this pattern are worth knowing:

- **All-local:** a single strong local model (30–32B) does both planning and execution, which is simplest to set up and is the right default for a laptop or workstation with one GPU.
- **Local planner, local executor:** a 30–32B planner drafts each step and a 7–8B executor applies it, trading some plan quality for throughput — worthwhile mainly on hardware that cannot comfortably hold two copies of a 32B model at once.
- **Cloud planner, local executor:** a frontier cloud model plans and a local model executes, which keeps the bulk of file contents on your own machine while still using strong reasoning for the decisions that matter most. This is a hybrid rather than a fully local setup — see the LiteLLM fallback pattern in Section for one way to wire a cloud-with-local-fallback endpoint, which composes with this split.

None of these routing choices substitutes for the guardrails below. A well-chosen planner still hands off to an executor that can make a per-step mistake, and the loop still needs a way to catch that.

Guardrails for autonomy

! Errors compound in an unattended loop

A frontier cloud model makes fewer per-step mistakes than a small local one, but the risk that matters here is not the per-step error rate on its own — it is that **autonomous** mode removes the human who would otherwise catch a mistake before it becomes the premise for the next ten steps. Structure the loop so that a mistake is caught by something other than a human watching in real time, or do not leave a small model fully unattended.

The mitigation for a higher per-step error rate is not a better model; it is a loop that cannot silently drift far from a known-good state. Five patterns do most of the work, and each is deliberately mechanical rather than judgment-based — the whole point is that they do not depend on the model noticing its own mistake:

1. **Verification gates after every step.** Run something that actually exercises the change — the test suite, `quarto render`, a linter, `R CMD check` — after each edit, and treat a non-zero exit as a hard stop for that step, not a suggestion. The environment’s pass/fail signal is the judge, never the model’s own claim that it “should work now.”
2. **Capped blast radius.** Run the loop in an isolated Git worktree (see the [worktree](#) feature), and commit after every step that passes its gate — never after a batch of several. A commit-per-green-step history means the worst outcome of a bad step is one commit to roll back, not an unreviewable pile of changes.
3. **Bounded loops.** Cap the total number of iterations and, separately, the number of *consecutive* failed gates. Hitting either cap should stop the loop and report what it tried, not retry indefinitely — a model that fails the same gate three times in a row is not going to succeed on the fourth attempt without a different approach, and a different approach is a decision for a human to make.
4. **Decomposition into specified, testable units, done up front.** Break the work into steps that each have a checkable definition of done before the loop starts, rather than handing the model one large, open-ended goal. A step with a clear pass/fail test is exactly the shape a small model handles well; an open-ended goal is exactly the shape that invites drift.
5. **Full logging.** Keep every prompt, tool call, and gate result the loop produced, so a run that stopped (or that a human later distrusts) can be reviewed after the fact rather than re-run blind.
6. **A tool surface small enough for the model.** Every tool the harness exposes costs context before the task starts, because its schema is sent with the prompt, and a configured

Model Context Protocol (MCP) server can add thousands of tokens of definitions on its own. Against a 32k local context that overhead competes directly with the work, and a small model loses: asked to read one file, a 7B model with a full MCP surface loaded produced a paragraph about missing credentials instead, and with MCP disabled invented a task list copied from the description of a tool it had been shown. Expose the smallest set of tools the step actually needs, and prefer a light harness over a heavyweight one — the same model that failed those attempts fixed a real bug and committed it in 17 seconds through **aider**.

These are the guardrails as a reader-facing rationale. The concrete gate wiring — an **ai-config** skill that launches a capped, worktree-isolated local loop, and a **gha** reusable workflow that runs one against a pull request using this repository’s own lint, spellcheck, and render checks as its gates — is tracked separately; see [Companion work](#) below.

Stack-specific notes

This site’s own stack — R, Python, Quarto, Julia, GitHub Actions YAML, and Markdown — is largely about producing *verifiable* artifacts: a script that runs, a document that renders, a workflow that passes. That is exactly the property that makes a narrow, checkable sub-task safe for an autonomous small-model loop, but the safety margin is not the same across languages:

Language / format	Autonomy dial	Notes
Python, Markdown, GitHub Actions YAML	Loosest leash	Well represented in training data; a syntax or lint check is a strong gate on its own
R	Tighter gates	Watch for non-tidyverse idioms and unfamiliar use of S4 or Reference (R5) classes; a model trained mostly on Python code can default to non-idiomatic R
Quarto (.qmd)	quarto render as the pass/fail judge	Have the model edit a known-good <code>_quarto.yml</code> rather than authoring one from scratch — a render failure is a strong, cheap gate
Julia	Shortest leash, strongest model, tightest test gate	The weakest training coverage of this stack’s languages, so treat any unattended Julia change as higher risk by default

None of this changes the guardrails above; it changes how tightly you set them — a smaller step size, a lower consecutive-failure cap, or simply keeping a human in the loop for Julia while letting a Markdown fix run unattended.

Fine-tuning: closing the idiom gap

A local model’s non-idiomatic R or Julia, noted in the stack-specific table above, is a training-data problem rather than a capability problem: the model has seen far less R and Julia than Python, not that it is incapable of writing either. Two lighter options are worth trying before fine-tuning anything:

- **Retrieval**, giving the model your own package’s existing R or Julia code as context so it has concrete idiom to imitate.
- **Prompting**, stating the conventions directly — this repository’s own `CLAUDE.md` and `.github/copilot-instructions.md` are examples of exactly that.

When those are not enough, **LoRA** (Low-Rank Adaptation) and its 4-bit variant **QLoRA** are the standard way to close an idiom gap without retraining a whole model. Both freeze the pretrained weights and train a small set of additional low-rank matrices on top, which cuts the trainable parameter count by orders of magnitude compared to full fine-tuning (Hu et al. 2021). QLoRA adds 4-bit quantization of the frozen weights on top of that, which is what actually shrinks the memory footprint enough to fine-tune a mid-sized model on a single consumer GPU (Dettmers et al. 2023). [Hugging Face’s PEFT library](#) is the common tooling entry point; the specifics of running it against this lab’s own repositories belong in `ai-config`, not here.

Whatever you fine-tune on, keep a held-out evaluation set of real tasks from your own codebase that the training data never touched, and re-check it after every fine-tuning run — a model that has memorized its training examples will look better on paper than it performs on the next genuinely new task.

Companion work

This page explains the reasoning; it does not implement a launcher or a CI gate. Two companion issues carry the runnable parts, each linking back here for rationale:

- [ai-config #1292](#): a skill that configures and launches a local autonomous loop — model choice, an Ollama or `llama.cpp` endpoint, and the guardrail caps above.
- [gha #436](#): a reusable workflow, a sibling to this repository’s own `claude.yml`, that runs a small/self-hosted-model agent against a pull request, wiring this site’s existing checks as the loop’s verification gates:
 - spellcheck
 - link check

- non-standard-characters
- bibliography DOIs

Configuring GitHub Copilot Settings

GitHub Copilot offers numerous configuration options that control how the AI assistant integrates into your development workflow. This section explains the key settings visible in your GitHub account preferences and provides guidance on which options to enable based on your use case.

Model Selection Options

GitHub Copilot provides access to multiple AI models, each with different capabilities and performance characteristics. The available models as of early 2026 include:

Anthropic Claude Models:

- **Claude Opus 4.1:** Most capable model for complex reasoning and analysis
 - *Pros:* Excellent at understanding nuanced requirements, handling complex codebases, superior code quality
 - *Cons:* Slower response times, may be overkill for simple tasks, limited availability (select option required)
 - *When to use:* Complex refactoring, architectural decisions, thorough code reviews
- **Claude Opus 4.5:** Latest version with enhanced capabilities
 - *Pros:* State-of-the-art performance, improved reasoning over 4.1
 - *Cons:* Similar trade-offs to Opus 4.1, requires selection
 - *When to use:* Most demanding tasks requiring cutting-edge capabilities
- **Claude Sonnet 4:** Balanced model optimizing capability and speed
 - *Pros:* Fast responses, strong performance, good default choice
 - *Cons:* Slightly less capable than Opus models for very complex tasks
 - *When to use:* General development work, most coding tasks
- **Claude Sonnet 4.5:** Enhanced version of Sonnet
 - *Pros:* Improved over Sonnet 4 while maintaining speed
 - *Cons:* Still not as powerful as Opus for extremely complex scenarios
 - *When to use:* Most daily development tasks
- **Claude Haiku 4.5:** Fast, efficient model for simpler tasks
 - *Pros:* Very fast responses, cost-effective, good for quick questions

- *Cons*: Less capable for complex reasoning or large codebases
- *When to use*: Simple completions, quick questions, repetitive tasks

OpenAI GPT Models:

As of 2026-05-08, OpenAI points users to ChatGPT Codex at chatgpt.com/codex. The OpenAI quickstart describes Codex as an AI coding assistant available through the Codex IDE extension that can read files, run commands, and write changes. This quickstart guide also links to a dedicated Codex app for working with local projects: [OpenAI Codex quickstart guide](#). For additional background and platform context, Wikipedia describes Codex as an AI coding agent by OpenAI with desktop app availability on Windows and macOS as an additional access path: [Codex \(AI agent\)](#). In the GitHub Copilot model names shown below, the **-Codex** suffix identifies code-specialized variants (for example, **GPT-5.2-Codex** and **GPT-5-Codex**).

- **GPT-5.2-Codex**: Specialized for code generation
 - *Pros*: Strong code completion, good at common patterns
 - *Cons*: May hallucinate package names or APIs
 - *When to use*: Code completion, common coding patterns
- **GPT-5**: Latest general-purpose model
 - *Pros*: Broad knowledge, good general performance
 - *Cons*: Not specifically optimized for code
 - *When to use*: Mixed tasks involving code and documentation
- **GPT-5-Codex** (various versions including Mini and Max):
 - *Pros*: Specialized variants for different use cases
 - *Cons*: Fragmented options can be confusing
 - *When to use*: Specific scenarios where variant optimizations matter

Connecting Positron Assistant to OpenAI

If you are using Positron Assistant with OpenAI models, set up an OpenAI API key first.

Follow these steps:

1. Go to the [OpenAI API keys page](#).
2. Sign in, choose or create the OpenAI project you want to use, and select **Create new secret key**.
3. Copy the key immediately and store it in a secure password manager.
4. In Positron, open the Command Palette with **Cmd+Shift+P** (or **Ctrl+Shift+P** on Windows/Linux).
5. Run **Positron Assistant: Configure Language Model Providers**.
6. Select **OpenAI**, paste your API key, and complete sign-in.

The [Positron Assistant getting started guide](#) states that OpenAI is enabled by default. If OpenAI does not appear as a provider, update Positron and confirm `positron.assistant.provider.openAI.enable` is not disabled.

Sources: [Positron Assistant setup](#), [OpenAI API key help](#), [OpenAI quickstart](#).

Google Gemini Models:

- **Gemini 2.5 Pro:** High-capability model
 - *Pros:* Strong multimodal capabilities, good at understanding context
 - *Cons:* Less proven in coding scenarios than Claude or GPT
 - *When to use:* Tasks involving images or complex context
- **Gemini 3 Pro/Flash** (Preview): Latest generation
 - *Pros:* Cutting-edge capabilities, flash variant offers speed
 - *Cons:* Preview status means less stable, limited track record
 - *When to use:* Experimental workflows, evaluation of new capabilities

Lab Recommendation: For most lab work, enable **Claude Sonnet 4.5** as your default model. It provides excellent balance of capability and speed. Consider switching to **Claude Opus 4.5** for complex architectural decisions or difficult debugging sessions. Keep **Claude Haiku 4.5** enabled for quick inline completions.

Feature Settings

These settings control where and how Copilot integrates into your development environment:

Editor preview features:

- *What it does:* Enables previews of experimental features in your editor
- *Pros:* Access to latest capabilities before general release
- *Cons:* May have bugs or unstable behavior
- *Recommendation:* **Enable** if you're comfortable troubleshooting issues and want cutting-edge features

Copilot Chat in GitHub.com:

- *What it does:* Enables Copilot chat interface on GitHub.com
- *Pros:* Quick access to Copilot without opening an editor, useful for reviewing PRs
- *Cons:* Only available with paid license
- *Recommendation:* **Enable** (included in GitHub Copilot subscription)

Copilot CLI:

- *What it does:* GitHub Copilot for assistance in terminal

- *Pros:* AI help for command-line operations, shell commands, and git operations
- *Cons:* Requires separate installation and setup
- *Recommendation:* **Enable** and install via `gh extension install github/gh-copilot`

Copilot in GitHub Desktop:

- *What it does:* Enables Copilot in GitHub Desktop app
- *Pros:* AI assistance in GUI git client
- *Cons:* Limited compared to editor integration
- *Recommendation:* **Enable** if you use GitHub Desktop

Copilot Chat in the IDE:

- *What it does:* Enables chat interface in your code editor
- *Pros:* Context-aware help, refactoring assistance, code explanation
- *Cons:* Can be distracting if overused
- *Recommendation:* **Enable** (essential feature)

Copilot Chat in GitHub Mobile:

- *What it does:* Enables Copilot chat in mobile app
- *Pros:* Quick access on mobile devices
- *Cons:* Limited by mobile interface
- *Recommendation:* **Enable** for convenience

Copilot can search the web:

- *What it does:* Allows Copilot to search internet for up-to-date information
- *Pros:* Access to current documentation, recent library changes, latest best practices
- *Cons:* May introduce latency, results depend on search quality
- *Recommendation:* **Enable** for access to current information

Advanced Settings

Dashboard Entry Point:

- *What it does:* Allows instant chatting when landing on GitHub.com
- *Pros:* Quick access to Copilot without navigating menus
- *Cons:* None significant
- *Recommendation:* **Enable** for convenience

Copilot code review:

- *What it does:* Use Copilot to review your code and generate pull request summaries
- *Pros:* Automated code review suggestions, PR summary generation, catches common issues

- *Cons:* May generate false positives, shouldn't replace human review
- *Recommendation:* **Enable** (major productivity boost)

Automatic Copilot code review:

- *What it does:* Automatically reviews all pull requests you create
- *Pros:* Catches issues early without manual triggering
- *Cons:* May be noisy on simple PRs, uses API quota
- *Recommendation:* **Disable** initially; enable only after you're comfortable with code review quality

Copilot coding agent:

- *What it does:* Delegate tasks to Copilot coding agent in repositories where it is enabled
- *Pros:* Autonomous multi-file edits, can execute complex refactoring, runs tests and fixes issues
- *Cons:* Requires careful oversight, can make unwanted changes if instructions unclear
- *Recommendation:* **Enable** (see Section for safe usage guidelines)

Copilot Memory (Preview):

- *What it does:* Remember repository context across Copilot agent interactions
- *Pros:* Better context awareness, learns repository patterns and conventions
- *Cons:* Preview feature governed by pre-release terms, potential privacy implications
- *Recommendation:* **Enable** to help Copilot learn your codebase patterns

MCP servers in Copilot:

- *What it does:* Connect MCP servers to Copilot in all editors and Coding Agent
- *Pros:* Extend Copilot with custom tools and integrations
- *Cons:* Requires MCP server setup and maintenance
- *Recommendation:* **Enable** if you have MCP servers configured; otherwise this setting has no effect

Copilot-generated commit messages:

- *What it does:* Allow Copilot to suggest commit messages when you make changes
- *Pros:* Saves time, generates descriptive messages based on code changes
- *Cons:* May miss important context, still requires review
- *Recommendation:* **Enable** but always review and edit suggested messages

Copilot Spaces:

- *What it does:* View and create Copilot Spaces (collaborative AI environments)
- *Pros:* Share AI context with team members
- *Cons:* Additional complexity for individual work

- *Recommendation:* **Enable** for team collaboration features

Copilot Spaces Individual Access:

- *What it does:* Create individually owned Copilot Spaces
- *Pros:* Personal AI workspaces for complex projects
- *Cons:* May fragment your workflow
- *Recommendation:* **Enable** for flexibility

Copilot Spaces Individual Sharing:

- *What it does:* Share individually owned Copilot Spaces
- *Pros:* Collaborate while maintaining ownership
- *Cons:* None significant
- *Recommendation:* **Enable** for sharing capability

Summary of Recommended Settings

For lab members, we recommend the following configuration:

Enable these features:

- All Copilot Chat options (GitHub.com, CLI, IDE, Mobile)
- Web search capability
- Dashboard Entry Point
- Copilot code review (but not automatic review initially)
- Copilot coding agent
- Copilot Memory
- MCP servers (if configured)
- Copilot-generated commit messages
- All Copilot Spaces options

Model selection:

- Default: Claude Sonnet 4.5
- Complex tasks: Claude Opus 4.5
- Quick completions: Claude Haiku 4.5

Enable with caution:

- Editor preview features (only if comfortable with potential instability)
- Automatic Copilot code review (wait until familiar with review quality)

Following these guidelines will help establish an effective Copilot configuration. The key is to enable features that add value to your workflow while maintaining awareness that AI assistance requires validation (see Section).

Connecting VS Code to a Custom Model Endpoint (BYOK)

VS Code's built-in Chat usually talks to GitHub's hosted models. It can also route requests to a model provider of your own; GitHub calls this "bring your own key" (BYOK). The lab uses BYOK to reach Databricks model serving endpoints, which expose an OpenAI-compatible API, through the community extension [oai-compatible-copilot](#).

This section describes the wiring and four errors you are likely to hit.

Wiring the extension to Databricks

Databricks serves models over an OpenAI-compatible endpoint at `https://<workspace>.cloud.databricks.com/serving-endpoints`. Point the extension at it in VS Code `settings.json`:

```
"oaiCopilot.baseUrl": "https://<workspace>.cloud.databricks.com/serving-endpoints",
"oaiCopilot.models": [
  {
    "id": "databricks-claude-opus-5",
    "owned_by": "databricks",
    "family": "claude",
    "context_length": 64000,
    "max_tokens": 16000,
    "delay": 15000,
    "vision": true,
    "apiMode": "openai"
  },
  {
    "id": "databricks-gpt-5-4",
    "owned_by": "databricks",
    "family": "gpt-5.4",
    "context_length": 64000,
    "max_tokens": 16000,
    "delay": 15000,
    "reasoning_effort": "medium",
    "vision": true,
    "apiMode": "openai"
  },
  {
    "id": "databricks-gpt-5-3-codex",
    "owned_by": "databricks",
    "family": "gpt-5.3-codex",
    "context_length": 64000,
```

```

    "max_tokens": 16000,
    "delay": 15000,
    "reasoning_effort": "high",
    "vision": true,
    "apiMode": "openai-responses"
  }
]

```

The `id` of each model must exactly match the name of a deployed serving endpoint in your workspace. The extension sends `id` as the OpenAI `model` field, and Databricks routes the request to the endpoint of that name. Most entries use `POST /serving-endpoints/chat/completions`. Models marked as Responses-API-only in the [Databricks model catalog](#), including GPT-5.3 Codex, instead require `apiMode: "openai-responses"`. An `id` that names no real endpoint fails (see the 404 below).

To store your token, run **Set OAI Compatible Multi-Provider Apikey** from the Command Palette (`Cmd+Shift+P` / `Ctrl+Shift+P`), choose the `databricks` provider, and paste a Databricks personal access token. The extension keeps it in VS Code's encrypted secret storage (under `oaicopilot.apiKey.databricks`) and sends it as an `Authorization: Bearer` header.

Sizing context and output limits

The model metadata controls both the request and the amount of conversation history Copilot sends:

- `context_length` is the total context window that the extension advertises to Copilot. It may deliberately be smaller than the provider's maximum window.
- `max_tokens` is the output cap. The extension maps it to `max_output_tokens` in Responses mode.
- `family` selects the closest Copilot system-prompt family.
- `vision` tells Copilot whether it may send images.

The extension advertises input capacity as `context_length` minus `max_tokens`. For example, `context_length: 64000` with `max_tokens: 16000` allows Copilot to send about 48,000 input tokens. Use the full provider window only when the workspace quota can sustain repeated agent turns at that size.

The underlying model's maximum output is not always a good value for `max_tokens`. [Databricks reserves the requested output allowance before admitting a request](#). Under the standard pay-per-token quota, Claude, GPT-5, and Gemini models generally have a 20,000 output-token-per-minute limit. A 64,000-token request can therefore receive an immediate 429 response even when the underlying model supports that output length.

Use the quota-aware lab defaults in Table 3. The context column is the operational value to put in `settings.json`, not the model’s maximum capability.

Table 3: Lab defaults for Databricks-hosted models

Model group	Workspace ITPM / OTPM	OAICopilot context	Output cap	Delay
GPT-5.6 Sol/Terra/Luna	2,000,000 / 200,000	400,000	16,000	0 ms
Claude Opus/Son- net/Haiku	200,000 / 20,000	64,000	16,000	15,000 ms
GPT-5.5 through GPT-5	200,000 / 20,000	64,000	16,000	15,000 ms
Gemini	200,000 / 20,000	64,000	16,000	15,000 ms
Inkling	200,000 / 10,000	64,000	8,192	15,000 ms
GPT OSS 120B/20B	1,000,000 / 100,000	131,072	25,000	0 ms
Llama 4 Maverick	1,000,000 / 100,000	128,000	8,192	0 ms
Llama 3.3/3.1 and Gemma 3	1,000,000 / 100,000	128,000	8,192	0 ms

Use `openai-responses` for GPT-5.3 Codex and any other endpoint that the current catalog marks as Responses-API-only. Use `openai` for the other models in the table. Add the `delay` value to each affected model entry. The extension applies this model-specific pause between requests; models without it fall back to the global `oaicopilot.delay` value. Every context and output cap in the table is a working default, not the underlying model’s maximum capability. Workspaces with higher provisioned or priority limits can raise these after checking their actual quota.

The 64,000/16,000 combination limits one prompt to about 48,000 input tokens. Together with 15-second pacing, that keeps one busy client near rather than far above a 200,000 ITPM tier. It is not a guarantee: the quota is shared across the workspace, prompt sizes vary, and concurrent users or clients consume the same allowance. Increase the delay or reduce `context_length` further when 429s continue.

Use longer retry spacing than the extension’s one-second default:

```
"oaicopilot.retry": {
  "enabled": true,
  "max_attempts": 3,
  "interval_ms": 15000,
  "status_codes": []
}
```

The extension already retries 429 responses and doubles this base interval on successive attempts. The longer starting interval gives Databricks’ sliding token window time to recover.

For GPT-5 and GPT OSS models, `reasoning_effort` adds a selector to the Copilot model configuration and is forwarded to Databricks. Start with `medium` for general work and `high` for Codex or difficult agentic tasks. Claude and Gemini 2.5 use provider-specific thinking controls; do not copy `reasoning_effort` onto those entries.

Databricks retires model endpoints over time. Before sharing or troubleshooting a configuration, compare every `id` with the current model catalog and remove entries that are no longer listed. An accurate local entry cannot make a retired endpoint work.

Four errors, and why they stack

These failures sit on top of each other: fixing one uncovers the next, so work through them top-down.

1. “No utility model is configured for ‘copilot-utility-small’ ”

When your main Chat model is a BYOK model, VS Code still needs a small “utility” model for background chores such as generating the conversation title and naming git branches. If none is configured, Chat fails before it ever reaches your provider:

```
No utility model is configured for 'copilot-utility-small'
while the selected main agent model is BYOK.
```

Set `chat.byokUtilityModelDefault` in `settings.json`:

- `"mainAgent"`: reuse your BYOK main model for these chores. This keeps all traffic on your provider and needs no extra endpoint, so it is the simplest choice.
- `"copilot"`: use GitHub’s hosted utility model. This needs an active Copilot subscription.
- `"none"`: the default, which errors on purpose.

This requirement arrived in a mid-2026 VS Code update. Before that, BYOK chat worked without the setting, so an editor update can make a working setup start failing here.

2. [404] ENDPOINT_NOT_FOUND

[404] Not Found

```
{"error_code": "ENDPOINT_NOT_FOUND",  
  "message": "The given endpoint does not exist, please retry after  
              checking the specified model and version deployment exists."}
```

The `model` name in the request is not a serving endpoint that exists in the workspace. Check that every `id` in `oaicopilot.models` matches a real, deployed endpoint (Databricks workspace → **Serving**), and remove or rename any entry that points at a name with no deployment. A stray placeholder entry, such as a leftover `copilot-utility-small`, is a common cause.

3. [403] Invalid access token

[403] Forbidden

```
{"error_code": 403, "message": "Invalid access token."}
```

The stored token is expired or revoked. Databricks OAuth tokens are short-lived and can expire within the day, so a session that worked in the morning can start returning 403 by afternoon; personal access tokens last until their configured expiry. Generate a fresh token (Databricks → **Settings** → **Developer** → **Access tokens**) and re-run **Set OAI Compatible Multi-Provider Apikey**. Prefer a long-lived personal access token to avoid frequent re-authentication. No window reload is needed; the extension reads the token on each request.

4. [429] REQUEST_LIMIT_EXCEEDED

[429] Too Many Requests

```
REQUEST_LIMIT_EXCEEDED: Exceeded workspace input tokens per minute rate limit  
for databricks-claude-sonnet-5.
```

This example is an input-tokens-per-minute (ITPM) failure. Lowering only `max_tokens` does not fix it: Databricks counts the actual prompt and conversation history against ITPM, while `max_tokens` reserves output-tokens-per-minute (OTPM) capacity.

For an ITPM error:

1. lower the model's operational `context_length`;
2. add or increase its model-specific `delay`;
3. lengthen `oaicopilot.retry.interval_ms`;
4. start a new conversation when accumulated history is no longer useful;
5. use GitHub's utility model for background chores when your Copilot plan allows it; and
6. ask the Databricks account team for a higher tier, or use provisioned throughput for sustained workloads.

For an OTPM error, lower `max_tokens` first. For either type, the error can persist until the sliding rate-limit window recovers.

Tip

A quick way to tell 404 from 403: a 404 means the request authenticated but named a missing endpoint (a model-name or configuration problem), while a 403 means the token itself was rejected (an authentication problem).

Configuring the Agent Environment

The `.github/workflows/copilot-setup-steps.yml` file allows you to customize the development environment in which the GitHub Copilot coding agent operates. This file preinstalls tools and dependencies so that Copilot can build, test, and lint your code more reliably.

Why Configure the Environment?

While Copilot can discover and install dependencies through trial and error, this can be slow and unreliable. Additionally, Copilot may be unable to access private dependencies. Preconfiguring the environment ensures:

- Faster agent startup and execution
- More reliable builds and tests
- Access to private or authenticated dependencies
- Consistent development environment across all agent sessions

File Location and Structure

The workflow file must be located at `.github/workflows/copilot-setup-steps.yml` in your repository's **default branch**. It follows GitHub Actions workflow syntax but must contain a single job named `copilot-setup-steps`.

Basic Configuration Example

See this repository's own [.github/workflows/copilot-setup-steps.yml](#) for a configuration adapted for R and Quarto projects.

Using actions/checkout

The [actions/checkout](#) action is used to check out your repository code so that the workflow can access it. While Copilot will automatically check out your repository if you don't include this step, **explicitly including it is necessary** when your setup steps need to access repository files.

Why explicitly include checkout?

Many dependency installation steps require access to repository files:

- `r-lib/actions/setup-renv@v2` needs `renv.lock` to install R package dependencies
- `r-lib/actions/setup-r-dependencies@v2` needs `DESCRIPTION` to install R package dependencies
- `npm ci` needs `package-lock.json` to install Node.js dependencies
- `pip install -r requirements.txt` needs the `requirements` file

Without an explicit checkout step, these dependency installation commands will fail because the necessary files won't be available yet.

Basic checkout:

```
- name: Checkout code
  uses: actions/checkout@v4
```

Important: The Copilot coding agent overrides any `fetch-depth` value you set in the checkout step. According to [GitHub's official documentation](#), this override happens “to allow the agent to rollback commits upon request, while mitigating security risks.” The agent dynamically determines the appropriate fetch depth based on the pull request context.

While you cannot control the fetch depth used by Copilot, the agent still has access to sufficient git history to perform its work effectively, including comparing changes and understanding the context of your pull request.

Configurable Options

You can customize only these specific settings in the `copilot-setup-steps` job:

- `steps`: Setup commands and actions to run
- `permissions`: Access permissions (typically `contents: read`)
- `runs-on`: Runner type (Ubuntu x64 Linux only)
- `services`: Database or service containers
- `snapshot`: Save environment state
- `timeout-minutes`: Maximum 59 minutes

All other workflow settings are ignored by Copilot.

Common Setup Tasks

For Node.js/TypeScript projects:

```
- name: Set up Node.js
  uses: actions/setup-node@v4
  with:
    node-version: "20"
    cache: "npm"

- name: Install dependencies
  run: npm ci
```

For Python projects:

```
- name: Set up Python
  uses: actions/setup-python@v5
  with:
    python-version: "3.11"

- name: Install dependencies
  run: pip install -r requirements.txt
```

For R projects:

```
- name: Set up R
  uses: r-lib/actions/setup-r@v2
  with:
    r-version: 'release'

- name: Install R dependencies
  uses: r-lib/actions/setup-renv@v2
```

Environment Variables and Secrets

To set environment variables for Copilot:

1. Navigate to your repository's **Settings**
2. Go to **Environments**
3. Select or create the `copilot` environment
4. Add environment variables or secrets as needed

Use secrets for sensitive values like API keys or passwords.

Testing Your Configuration

The workflow runs automatically when you modify `copilot-setup-steps.yml`, allowing you to validate changes in pull requests. You can also manually trigger the workflow from the repository's **Actions** tab.

Setup logs appear in the agent session logs when Copilot starts working. If a step fails, Copilot will skip remaining steps and begin working with the current environment state.

Advanced Configuration

Larger runners: For projects requiring more resources, you can use larger GitHub-hosted runners:

```
jobs:
  copilot-setup-steps:
    runs-on: ubuntu-4-core
```

Self-hosted runners (ARC): For access to internal resources or private registries, use Actions Runner Controller (ARC) self-hosted runners:

```
jobs:
  copilot-setup-steps:
    runs-on: arc-scale-set-name
```

Note: When using self-hosted runners, you must disable Copilot's integrated firewall in repository settings and configure appropriate network security controls.

Git Large File Storage (LFS): If your repository uses Git LFS:

```
- uses: actions/checkout@v4
  with:
    lfs: true
```

Further Reading

For complete details, see [Customizing the development environment for GitHub Copilot coding agent](#).

Agent Skills

[Agent Skills](#) are a lightweight, open standard for extending AI agent capabilities with specialized knowledge and workflows. The [specification](#) defines a portable, tool-agnostic format that any compatible agent can load.

What is a Skill?

At its core, a skill is a folder containing a `SKILL.md` file. This file includes the **name** and **description** metadata [required by the specification](#), along with instructions that tell an agent how to perform a specific task. Skills can also bundle supporting resources:

- Scripts
- Reference documentation
- Templates and assets

```
my-skill/
  SKILL.md          # Required: metadata + instructions
  scripts/          # Optional: executable code
  references/        # Optional: documentation
  assets/           # Optional: templates, resources
```

How Agents Load Skills

The specification describes a *progressive disclosure* model, in which an agent typically loads a skill in three stages:

1. **Discovery:** At startup, the agent loads only the name and description of each available skill — just enough to know when it might be relevant.
2. **Activation:** When a task matches a skill's description, the agent reads the full `SKILL.md` instructions into context.
3. **Execution:** The agent follows the instructions, optionally executing bundled scripts or loading referenced files as needed.

This means full instructions load only when needed, so agents can maintain many skills with a small context footprint.

Why Use Skills?

Skills package procedural knowledge and team-specific context into portable, version-controlled folders. This gives agents:

- **Domain expertise:** Capture specialized knowledge as reusable instructions
- **Repeatable workflows:** Turn multi-step tasks into consistent, auditable procedures
- **Cross-tool reuse:** Build a skill once and use it across any skills-compatible agent

Further Reading

For the complete specification and more details, see agentskills.io.

A skill is one of several ways to customize an agent, and not always the right one. Section compares it against:

- instruction files
- subagents
- hooks
- permissions

That section also explains why Claude Code’s custom slash commands are now skills themselves.

The [Morrison-Lab/ai-config](https://github.com/Morrison-Lab/ai-config) repository contains an example of personal Claude Code configuration, including user-level skills, hooks, and subagents, synced across machines via Git.

Customizing an Agent

Section describes one way to extend an agent. It is not the only one, and a lab that knows only that one tends to write every customization as a skill, including the ones that should have been something else.

This section maps the whole surface. The mechanisms differ less in what you can write in them — most are Markdown with a [YAML front matter](#) header, as Section describes — than in **when they fire and who decides**.

The Question That Picks the Mechanism

Ask two things about the behavior you want:

- **Who triggers it?** You, by typing something; the model, by judging it relevant; or the harness, on a fixed event.
- **What happens if the model disagrees?** Some mechanisms are advice the model may ignore. Others are enforced by the harness whatever the model decides.

That second question is the one people get wrong, and Claude Code’s own documentation is blunt about it. Instruction files are [described](#) as “context, not enforced configuration”, delivered “as a user message after the system prompt”, so “there’s no guarantee of strict compliance.” The same page names the remedy:

To block an action regardless of what Claude decides, use a `PreToolUse` hook instead.

A rule you cannot afford to have ignored does not belong in a `CLAUDE.md`.

Instruction Files: Always-On Context

`CLAUDE.md` and [AGENTS.md](#) are prose the harness loads at the start of a session, with no front matter and no schema (Section).

Three properties matter when you write one:

- **Files concatenate; they do not override.** Claude Code [walks up the directory tree](#) from the working directory, and “all discovered files are concatenated into context rather than overriding each other”, ordered from the filesystem root down. A project file does not replace your personal one.
- **Imports exist, and they skip code spans.** The `@path/to/import` syntax pulls in another file, recursively, to a maximum depth of four hops. Paths resolve relative to the importing file. To *mention* a path without importing it, wrap it in backticks — import parsing skips fenced code blocks and code spans.
- **The two filenames are not interchangeable.** Claude Code [reads CLAUDE.md](#), [not AGENTS.md](#), and recommends a `CLAUDE.md` whose first line is `@AGENTS.md` so both tools read one source. A symlink works on macOS and Linux; on Windows it needs Administrator privileges or Developer Mode, so the import is the portable choice. GitHub Copilot, by contrast, [reads all of AGENTS.md](#), `CLAUDE.md`, and `GEMINI.md`.

`AGENTS.md` is stewarded by the Agentic AI Foundation under the Linux Foundation, and it is worth being precise about what it standardizes: a filename, a location, and a nearest-file-wins precedence rule. Asked whether there are any required fields, its own FAQ answers that there are none — “AGENTS.md is just standard Markdown.” So two agents reading the same file are guaranteed to *see* the same text and guaranteed nothing about acting on it alike.

Workspace Rules and Activation Modes

Instruction files and rules can be scoped globally or to a specific workspace. Google Antigravity and Cursor extend basic instruction files by supporting explicit rule activation modes and workspace rules (`.agents/rules/` or `.cursor/rules/*.mdc`):

- **Always On:** Included unconditionally in every prompt context for the workspace (e.g. `GEMINI.md`, `AGENTS.md`, or `.mdc` files with `alwaysApply: true`).
- **Glob Scoped:** Activated automatically only when matching specific file patterns or paths in the workspace (e.g. `globs: ["src/ui/**/*"]` or Copilot's `applyTo`).
- **Model Decision:** Dynamically injected into context when the model judges the rule relevant to the current task or user prompt.
- **Manual:** Explicitly invoked by name or `@mention` during a session.

In Antigravity, workspace rules live under `.agents/rules/` (project-level) with global rules in `~/.gemini/GEMINI.md`, and workspace discovery operates via directory structures (`.agents/skills/` and `.agents/plugins/`).

Skills, and the Commands That Became Them

The distinction most people still draw here is out of date. Claude Code's documentation [states](#):

Custom commands have been merged into skills. A file at `.claude/commands/deploy.md` and a skill at `.claude/skills/deploy/SKILL.md` both create `/deploy` and work the same way.

Existing `.claude/commands/` files keep working, and if a command and a skill share a name, the skill wins. So “slash command versus skill” is now a question about **an older file layout versus a newer one**, not about two different capabilities.

What did *not* collapse is the invocation question. It moved into front matter, where it is now set per skill rather than implied by which directory the file sits in:

Front matter	You can invoke	The model can invoke
<i>(default)</i>	yes	yes
<code>disable-model-invocation: true</code>	yes	no
<code>user-invocable: false</code>	no	yes

This is the setting to think hardest about. A skill the model cannot invoke will never fire unless someone remembers it exists; a skill the model *can* invoke costs context on every turn, because its description sits in the listing whether or not it is ever used. Note also that `user-invocable` controls menu visibility rather than access: to block programmatic invocation, use `disable-model-invocation`.

The portable core is small and worth knowing exactly. The [Agent Skills specification](#) requires precisely two front matter fields — **name** and **description** — and says of the body that there are no format restrictions. Everything past that is a vendor extension: Claude Code’s own docs describe invocation control, subagent execution, and dynamic context injection as extensions to the standard. Portability is therefore real but shallow: the folder and its metadata travel, and how much of the *behavior* travels depends on how alike two agents happen to be.

One concrete sign that the format genuinely crosses vendors: GitHub Copilot [looks for skills](#) in `.github/skills`, `.agents/skills`, and `.claude/skills` — a competitor’s directory name.

Subagents: Delegating to a Fresh Context

Section and Section cover what a subagent *is* and how the harness runs one. The authoring question is narrower: a subagent is a single Markdown file with front matter in `.claude/agents/` (project) or `~/.claude/agents/` (personal), whose body becomes that agent’s entire system prompt.

Two details are easy to get wrong.

Precedence runs the opposite way from skills. For subagents, a project definition [outranks](#) a personal one. For skills, personal [outranks](#) project. If you keep a personal copy of something the repository also defines, which one wins depends on which mechanism you chose.

An @-mention picks the worker, not the words. Naming a subagent guarantees which one runs. It does not hand that subagent your sentence:

Your full message still goes to Claude, which writes the subagent’s task prompt based on what you asked. The @-mention controls which subagent Claude invokes, not what prompt it receives.

Hooks: The Part the Model Cannot Talk Its Way Around

Hooks are the mechanism this manual has not previously covered, and the one that changes what a customization is *worth*. They are [defined in JSON settings files](#) rather than in Markdown, and they run as shell commands, HTTP calls, or LLM prompts at fixed points in the harness’s lifecycle. The [page on instruction files](#) draws the contrast plainly:

Hooks execute as shell commands at fixed lifecycle events and apply regardless of what Claude decides to do.

An event fires, a matcher selects which handlers apply (by tool name, for instance), and the harness passes the handler JSON describing the event. Documented events include **SessionStart**, **UserPromptSubmit**, **PreToolUse**, **PostToolUse**, **Stop**, and **SessionEnd**, among a longer list; consult the reference rather than this page for the current set.

The practical rule: anything stated as “always” or “never” in a `CLAUDE.md` is a candidate to become a hook. Prose asks; a `PreToolUse` hook decides.

Settings and Permissions: The Boundary Around All of It

Settings live in `~/.claude/settings.json` (user), `.claude/settings.json` (project, shared), and `.claude/settings.local.json` (project, private), with a managed-policy layer above them for organizations. The [precedence](#) runs managed, then command-line arguments, then local, then project, then user.

One exception deserves emphasis, because it is the opposite of what the ladder implies:

Permission rules behave differently because they merge across scopes rather than override.

Rules are written as `Tool(specifier)` — for example `Bash(npm run test *)`, `Read(./env)`, or `Skill(commit)` — and sorted into `allow`, `ask`, and `deny`. Because a project’s rules merge with yours rather than replacing them, a repository can tighten what you allow, and cannot quietly loosen it.

MCP Servers Are a Different Axis Entirely

Everything above changes what the agent *knows or must do*. An [MCP](#) server changes what it *can reach*: typed tools, data resources, and reusable templates exposed over a standard protocol. The specification is explicit that it “does not dictate how AI applications use LLMs or manage the provided context.”

So MCP is never the answer to “how do I make the agent follow our convention”, and always a candidate answer to “how do I let the agent query our issue tracker”. Section [covers](#) configuration and its failure modes.

Choosing

If you want to...	Use	Fires when
State a convention that should color everything	<code>CLAUDE.md</code> / <code>AGENTS.md</code>	every session, as context
Package a procedure the model should notice on its own	a skill	the model judges it relevant
Package a procedure <i>you</i> will invoke by name	a skill with <code>disable-model-invocation</code>	you type <code>/name</code>
Hand off self-contained work to a fresh context	a subagent	the model delegates, or you <code>@-mention</code>

If you want to...	Use	Fires when
Enforce something regardless of the model	a hook	a lifecycle event
Constrain what may run at all	permissions in settings	every tool call
Give the agent access to an external system	an MCP server	the model calls the tool

What Travels Between Tools

Mechanism	Portable?	Evidence
Agent Skills (SKILL.md)	yes, in format	open standard with a published specification, adopted across Claude Code, Gemini CLI, Codex, Copilot, and Cursor
AGENTS.md	yes, as an open specification	standardizes filename, location, and precedence across Codex, Gemini CLI, Cursor, Aider, and Copilot (Claude Code reads CLAUDE.md by default, or imports @AGENTS.md)
MCP servers	yes	open protocol with multiple independent clients
CLAUDE.md / GEMINI.md	by courtesy	vendor instruction files read by default in their respective CLI environments, and supported by courtesy in GitHub Copilot
Cursor Rules (.cursor/rules/*.mdc)	no	Markdown Cursor (.mdc) files with <code>alwaysApply</code> and <code>globs</code> frontmatter, scoped to Cursor
.github/copilot-instructions.md .github/instructions/*.instructions.md	no	GitHub Copilot only
*.prompt.md prompt files	no	GitHub Copilot only, and not on every Copilot surface Copilot only , and “only available in VS Code, Visual Studio, and JetBrains IDEs”

Mechanism	Portable?	Evidence
Hooks, settings, permissions	no	each harness defines its own

The lesson for a lab is to keep the portable layer carrying the meaning. Conventions belong in `AGENTS.md` and skills, which survive a change of tool; hooks and permissions are worth writing, and are worth writing as enforcement of rules that are also stated somewhere portable.

A Worked Example

The [Morrison-Lab/ai-config](#) repository is one lab member's configuration, versioned and synced across machines. Counting its `main` branch on 4 August 2026 — `git ls-tree -d --name-only origin/main skills/ | wc -l`, and the equivalent for the other directories — it carries:

- 177 skill directories
- 1 file under `commands/`, from before the merge described above
- 21 hook scripts, registered across `UserPromptSubmit`, `PreToolUse`, and `Stop`
- 7 subagent definitions

That distribution is itself the argument. Nearly everything is a skill, because a skill is the mechanism the model can reach for unprompted. The hooks are few and specific, because each one exists to make a rule that was being forgotten impossible to forget.

This repository is a smaller example of the same idea: it carries a `.github/copilot-instructions.md` for conventions that apply everywhere, plus path-scoped files under `.github/instructions/` whose `applyTo` globs attach them only when you edit a matching file.

How the Config Reaches a Machine

Section describes *which* mechanism a customization should use. This section is about the step after that decision: how a config like a shared instruction repository actually reaches a machine, and how a broken install fails.

Two agents can load the identical instruction corpus and still behave differently, because behavior depends not only on what the config says but on how it is installed where the agent runs. An install problem is quiet by construction — nothing errors, the work still gets done, and a capability simply goes missing with no message that it existed.

Two Ways the Same Config Reaches a Machine

A repository of skills, hooks, and instruction files can be delivered two ways, and they are alternatives rather than layers.

- **As a plugin.** You enable the repository as a plugin from a marketplace, and the harness loads its skills, commands, and hooks directly from the plugin's own checkout at session start. It refreshes when the marketplace updates.
- **As a per-machine install.** You symlink (or copy) the repository's children into `~/.claude/` — `skills/`, `shared/`, `memories/`, `hooks/`, and `CLAUDE.md` — and register the hooks into `~/.claude/settings.json` by hand or with a script.

The trap is running both at once. The plugin path and the settings-file path carry different command strings, so the harness keeps both, and every hook fires twice. This fails in the safe direction — a doubled guard warns or blocks twice, it never goes missing — which is exactly why it is easy to leave in place unnoticed. Pick one path.

The `~/.claude` Layout

On a symlink-capable system, the children of `~/.claude` are symlinks into a working checkout of the repository, so a `git pull` in that checkout refreshes every skill and rule for free. Windows Git Bash is the common exception: without symlink privileges configured (Developer Mode, or `MSYS=winsymlinks:nativestrict`), its `ln -s` falls back to real copies, so a pull does not propagate and the copies must be re-synced.

Two health checks answer two different questions, and a clean answer to one says nothing about the other.

- **Files.** Does `~/.claude/hooks/<script>` still track the checkout, or has it drifted, gone missing, or become a dangling link?
- **Bindings.** Does `settings.json` actually invoke that script on an event?

A guard can be a perfectly linked file that is registered to nothing, and an unregistered guard and a guard with nothing to block produce the same output: none. So verify both, and read the two results separately.

One Plugin Per Capability

Enabling the *same* repository as a plugin from two marketplaces — a personal fork and a lab org, say — loads its whole skill set into every session twice. That is not just untidy. A large instruction corpus is already a substantial share of a session's context, and a duplicated one can push a session far enough over the model's context limit to break subagent delegation, because a subagent independently re-loads that same duplicated skill set at start, and that alone can exceed the limit before any work begins. Keep one copy enabled.

Failure Modes and Their Symptoms

A broken install rarely announces itself; you read it backward from a symptom.

- **A stale hook reference blocks the tool it guards.** A `settings.json` entry pointing at a hook script that no longer exists errors before the guarded tool runs, so every call to that tool fails with the hook's error rather than the tool's output. A `PreToolUse` hook on `Bash` that references a missing script, for instance, blocks *all* shell commands until the entry is removed.
- **A dangling symlink silently drops a file.** A `~/.claude` child symlinked into a temporary worktree or a since-deleted clone becomes a dead link when that directory is removed, and the instructions or agent definition it provided quietly stop loading.
- **A check run against the wrong checkout cries wolf.** A verification that compares the installed copies against a stale or unintended checkout can report every entry as misdirected. That is a false alarm from a check aimed at the wrong target, not a fleet of real problems; re-run it against the checkout the install is actually anchored to before acting.

The common thread is that the install layer is a real surface, distinct from the content of the config, with its own failure modes and its own checks. When an agent behaves as though a rule or skill you wrote does not exist, suspect the install before you suspect the rule.

Claude Code Cloud Environments

[Claude Code](#) is a CLI coding agent that can also run tasks on Anthropic-managed cloud infrastructure— either from the web at [claude.ai/code](#) (“Claude Code on the web”), or from the terminal by adding the `--remote` flag to move a session into the cloud.

Each cloud run executes inside a configured **environment**. An environment bundles three things:

- **Network access level:** what the cloud session is allowed to reach (a security control, analogous to the firewall configuration described above).
- **Environment variables:** values the session needs at runtime, such as `NODE_ENV`, database URLs, or API keys.
- **Setup scripts:** Bash that runs automatically when the session starts, for example to install dependencies.

Selecting the default environment with `/remote-env`

The `/remote-env` slash command sets **which configured environment is the default** for `--remote` runs:

- With a single environment configured, it shows your current configuration.
- With multiple environments, it opens an interactive picker so you can choose the default.

`/remote-env` only *selects* the default; to add, edit, or archive the environments themselves, use the web interface at claude.ai/code. Because `/remote-env` opens an interactive panel, run it from an interactive `claude` terminal session.

Availability

Claude Code on the web (and the cloud environments it relies on) is a research-preview feature, available to Pro, Max, and Team users (and Enterprise users with eligible seats). Availability and behavior may change.

For details, see the [Claude Code on the web documentation](#) and the [slash command reference](#).

How a Session Learns a PR Changed

A coding-agent session that is watching a pull request does not poll it. Something wakes the session when the pull request changes, and in Claude Code that “something” is one of **two separate channels**.

The distinction matters for a practical reason: **the agent can turn one of them on and off, and cannot touch the other**. So “stop watching this PR” is a request an agent can satisfy for one channel and can only decline for the other.

Both arrive the same way — mid-turn, alongside the next tool result, the same delivery mechanism as a background task notification — so they are easy to mistake for each other. They are distinguishable by the tag wrapping them.

	<github-webhook-activity>	<ci-monitor-event>
Turned on by	the agent, via a tool call	a human, via a checkbox
Turned off by	the agent, via a tool call	a human, via the same checkbox
Available in	a session with the remote/hosted GitHub MCP server (not a local one)	Claude Code on the web only
Carries	comments, CI results, reviews, mergeability notices	the new comment, plus a fixed instruction template

The channel an agent controls

`subscribe_pr_activity` and `unsubscribe_pr_activity` are ordinary tool calls, taking an owner, a repository, and a pull-request number. Subscribing delivers that pull request’s activity into the conversation as `<github-webhook-activity>` messages until the session unsubscribes, or the pull request merges or closes.

Two caveats are worth knowing before relying on it.

A successful subscribe does not guarantee delivery. If a PR Steward agent already holds the watch on that pull request, the call still succeeds — but this session receives nothing. The tool result says so in as many words, so read the result rather than the exit status. Taking over the watch requires opting the steward out first, by removing its watching label on the pull request.

The tool does not exist on a locally-run GitHub MCP server. Workflow guidance written for remote or web sessions names it freely, which strands anyone following that guidance from a local harness. Section covers the local analogues to reach for instead.

Webhook delivery is also not exhaustive, which is the failure mode most likely to be mistaken for “nothing has happened”. CI *successes*, new pushes, and merge-conflict transitions can arrive late or not at all. A session that treats silence as “still green” will sit indefinitely on a pull request that has gone stale or conflicted, so a subscription is a supplement to periodically re-reading the pull request’s real state, not a replacement for it.

The channel only a human controls

Claude Code on the web shows a per-pull-request **CI monitoring** panel in the session sidebar, with two checkboxes: **Auto-fix CI & address comments** and **Auto-merge when ready**.

Ticking the first is what starts `<ci-monitor-event>` messages arriving.

Three properties of that panel surprise people:

- **There is no default.** No account-, organization-, repository-, or environment-level setting turns either checkbox on ahead of time. Absent the `/autofix-pr` shortcut described below, every new pull request and session starts with both off.
- **No agent-side tool can reach it.** It is client-UI state, not something an agent’s configuration surface touches, so asking an agent to enable it cannot work. If the checkbox changes, a human changed it.
- **One of the two has a shortcut, and the other does not.** Running `/autofix-pr` from the command line on a pull request’s branch spawns a web session with **Auto-fix CI & address comments** already on. There is no equivalent shortcut for **Auto-merge when ready**.

See Section for the web-session context these run in.

The instruction template is boilerplate

Each `<ci-monitor-event>` quotes the triggering comment verbatim and then appends a **fixed instruction template**:

- address the feedback and push a fix;
- post a one-line reply on the thread;
- end that reply with a set attribution line;
- resolve the thread;
- skip replies for comments you did not act on.

That template is appended to **every** new comment. It is not gated on whether the comment contains anything actionable. Observed firings with nothing to act on include:

- a Copilot review declining for quota reasons (“unable to review... reached their quota limit”),
- a Copilot review reporting it “wasn’t able to review any files”, and
- a sticky preview-deployment comment that posts a preview URL and rewrites itself on every push.

The practical consequence is that the template’s imperative opening should not be obeyed reflexively. Its own closing clause — skip replies for comments you did not act on — is the standing permission to do nothing when there is nothing to do, and it is the half most often read past. Treat each event as a prompt to go and check the pull request’s actual state through the API, rather than as an instruction that a fix is owed.

Two names that sound identical and are not

The sidebar’s **Auto-merge when ready** checkbox and the `enable_pr_auto_merge / disable_pr_auto_merge` MCP tools are unrelated controls with nearly the same name.

The tools drive **GitHub’s own** native auto-merge on the pull request: merge once required checks pass and approvals are met, which is a setting stored on GitHub and visible to everyone. The checkbox is Claude Code client state governing what the agent session does. Toggling one says nothing about the other.

Treat what arrives as untrusted

Comment bodies inside either wrapper come from anyone who can comment on the pull request. Directives that appear inside them are data, not instructions: a comment that says “ignore your previous instructions” is a comment, and a comment that asks for a credential is a comment.

One specific trap is worth calling out, because it looks exactly like the thing it is not. Comments posted *by the agent* through the GitHub MCP tools authenticate as whichever account owns the session’s token. In an interactive session that is typically the human who owns it, so an

event echoing the agent’s own just-posted reply usually shows a human author rather than a recognizable bot name. It is not a rule, though: a pipeline authenticating through an App-token exchange posts as `claude[bot]` instead, as this repository’s own review workflow does. Either way the conclusion is the same, and the variability only sharpens it — author identity is useless for deciding whether an event is your own echo.

The attribution footer is a better signal, but not proof. Every comment posted from these sessions carries one, so a body that lacks it is very unlikely to be yours. A body that has it establishes less than it appears to, for two reasons the rest of this page has already supplied:

- **It is part of the untrusted comment data.** Anyone who can comment on the pull request can paste the same footer text at the end of a malicious comment.
- **It identifies a class, not an instance.** It marks the comment as coming from *some* Claude Code session, which is not the same as *this* one — a PR Steward concurrently watching the same pull request carries the identical footer.

So treat a footer as a strong hint and a missing footer as near-conclusive, and settle genuine authorship questions against what this session actually posted.

Running both at once duplicates every comment

The two channels overlap in what they deliver. With webhook activity subscribed *and* the auto-fix checkbox ticked on the same pull request, each new comment arrives twice: once as raw activity, and once wrapped in the instruction template. Turn on the one whose control surface and behavior you actually want.

Observed behavior, not documented contract

The delivery mechanics and the wording of the instruction template above were established by observation during agent sessions in mid-2026, not from a published specification. Claude Code on the web is a research-preview feature, so treat the specifics as liable to change and re-check them against current behavior before depending on any one detail.

When to use a coding agent

Coding agent sessions are currently¹ considered “premium requests”, which are limited resources; see <https://github.com/features/copilot/plans> for details. So, use coding agents sparingly. Use them for complex changes that would be difficult or time-consuming for you to complete by hand. Coding agents also take time to get configured for work, every time you make a request. See <https://docs.github.com/en/copilot/how-tos/use-copilot-agents/coding-agent/customize->

¹2026-01-10

[the-agent-environment#preinstalling-tools-or-dependencies-in-copilots-environment](#) for ways to reduce that startup time, but it will never be 0. If you can complete the task faster than the coding agent can, you should probably do it yourself. For example, when you have errors in the spell-check or lint workflows, you can often fix them faster than Copilot can. Similarly, when reviewing Copilot’s PRs, you can often make direct changes to the branch faster than you could write clear review comments and get Copilot to address them.

Also, the less we practice, the weaker our skills get, and the harder it is for us to supervise the agents and make sure they are actually doing what we want them to do, the way we want them to do it. You should exercise your own coding skills regularly, just like you would for any other skill you want to maintain.

Editing with .docx files

GitHub Copilot coding agents can read Microsoft Word (.docx) files, including tracked changes and comments. This enables a hybrid editing workflow where:

1. Lab members can export Quarto content to Word format for review
2. Reviewers can make edits, add tracked changes, and insert comments in Word
3. Coding agents can read the .docx file and translate the edits back to Quarto format

When using this workflow, make sure to explicitly instruct the coding agent to:

- Examine and apply all tracked changes in the .docx file
- Read and address all comments in the .docx file
- Translate edits from Word formatting to appropriate Quarto/markdown syntax

This approach makes it easier for collaborators who are more comfortable with Word to contribute while maintaining the source files in Quarto format.

Known Issue: “Document 1” Warning in Word

When opening DOCX files generated by Quarto (including this site), Microsoft Word may display a warning message and open the file with the title “Document 1” instead of the actual filename. Word may also require you to save the file before you can add comments or track changes.

This is a known limitation with how Quarto generates DOCX files. The issue is being tracked in the Quarto project:

- [Quarto CLI Issue #6357](#)
- [Quarto Discussion #6544](#)
- [Quarto CLI Issue #10587](#)

Workaround: If you are the author generating the DOCX file from Quarto, follow these steps before sharing with collaborators:

1. Open the generated DOCX file in Microsoft Word
2. Immediately save the file (File → Save, or Ctrl+S/Cmd+S)
3. Close and re-open the file to verify it no longer shows “Document 1”
4. Share this saved version with collaborators

This one-time step ensures that when collaborators open the file, they won’t see the “Document 1” warning and can immediately add comments and track changes without issues.

Copilot Instructions for this Repository

A `.github/copilot-instructions.md` file contains repository-specific instructions and guidelines for GitHub Copilot coding agents. This file helps ensure that AI-generated contributions follow the project’s formatting standards, coding conventions, and documentation practices.

For a Quarto-based repository like this one, the copilot instructions file typically specifies:

- Markdown and Quarto formatting rules (e.g., blank lines before lists, line breaks in prose)
- R code style guidelines (e.g., using native pipe `|>`, following tidyverse style)
- File organization patterns (e.g., using Quarto includes for modular content)
- How to work with DOCX files for hybrid editing workflows
- Version control best practices (e.g., do not commit rendered artifacts that are generated only for preview/review)
- Repository-specific best practices

By having these instructions in `.github/copilot-instructions.md`, you ensure that coding agents produce consistent, high-quality contributions that align with the project’s established practices. This reduces the review burden and helps maintain consistency across all contributions, whether made by humans or AI assistants.

See this repository’s own [.github/copilot-instructions.md](#) for a working example.

Using Copilot Review Before Human Review

Before requesting review from other humans, **always have Copilot review your pull request first**—even if Copilot created the PR itself. AI review provides fast, thorough feedback that helps catch issues before involving human reviewers, saving everyone time and improving code quality.

Why review with Copilot first:

- **AI has more bandwidth:** Copilot can review code immediately without competing priorities
- **Catch common issues early:** Copilot excels at identifying bugs, logic errors, security vulnerabilities, and style inconsistencies
- **Improve human review quality:** When humans review cleaner code, they can focus on higher-level concerns like design and architecture rather than basic issues
- **Learn from feedback:** Even experienced developers benefit from Copilot’s perspective on best practices and potential improvements
- **Growing capabilities:** AI review capabilities continue to improve over time, making this investment increasingly valuable

Copilot review workflow:

1. **Assign Copilot as a reviewer:** On your pull request page, assign Copilot to review the PR the same way you would assign any other reviewer. Click “Reviewers” in the right sidebar and select Copilot from the list.
2. **Review Copilot’s comments:** Once Copilot completes its review, carefully examine each comment. For each comment, decide whether you agree with the suggestion:
 - **If the comment is correct:** Address it by making code changes yourself or ask Copilot to apply the fix using GitHub’s suggestion features
 - **If the comment is incorrect or not applicable:** Dismiss the comment with an explanation for why it doesn’t apply
 - **If you’re uncertain:** Seek a second opinion from a human reviewer or do additional research
3. **Request another Copilot review:** After addressing or dismissing all comments, request another review from Copilot. This creates an iterative improvement process.
4. **Iterate until satisfied:** Repeat the review-and-address cycle until Copilot stops providing valuable suggestions. This typically takes 1-3 iterations depending on the complexity of the changes.
5. **Request human review:** Only after you’ve addressed Copilot’s feedback should you request review from human team members. At this point, the code should be in better shape, allowing human reviewers to focus on higher-level concerns.

Important considerations:

- **Copilot isn’t perfect:** AI review can produce false positives or miss important issues. Always apply your own judgment when evaluating Copilot’s suggestions.
- **Don’t blindly accept all suggestions:** Some of Copilot’s recommendations may not fit your specific context or requirements. It’s perfectly appropriate to dismiss comments that don’t apply.

- **Human review remains essential:** Copilot review supplements but does not replace human code review. Humans bring domain knowledge, understanding of business requirements, and judgment about trade-offs that AI cannot replicate.
- **Document dismissals:** When dismissing Copilot comments, briefly explain why. This helps human reviewers understand your reasoning and can serve as documentation for future reference.

For pull request authors:

Even if you're highly experienced, treating Copilot review as a required pre-review step helps maintain code quality and makes the best use of everyone's time. The few minutes spent on Copilot review often save hours of back-and-forth with human reviewers.

For human reviewers:

When you receive a PR for review, check whether the author has completed the Copilot review process. If Copilot hasn't reviewed the PR yet, consider asking the author to complete that step first before you invest time in review. This ensures you're reviewing code that has already been through initial automated quality checks.

Reviewing a Copilot PR You Didn't Create

When reviewing a pull request where someone else prompted Copilot to make changes, follow these guidelines to avoid confusion and ensure smooth collaboration:

Understanding PR roles:

The general PR roles (issue creator, author, reviewer, merger, assignee, and PR steward) are described in the [UCD-SERG Lab Manual's GitHub chapter](#). Copilot-assisted workflows add two more:

- **PR prompter:** Assigns a developer (human or AI) to start working on a PR, often by assigning an issue to Copilot. The PR prompter is sometimes the same person as the issue creator, but is often a project maintainer who reviews and triages external issue reports. When Copilot creates commits, both Copilot and the prompter are listed as co-authors. Typically the PR prompter becomes the PR manager.
- **PR manager:** Supervises and guides the PR authors, assigns reviewers, and controls the PR workflow, deciding when and how Copilot makes additional changes. The PR manager can hand off this role to someone else.

The scenario:

- One team member (the "PR prompter") assigned Copilot to work on an issue or explicitly prompted Copilot to start working
- The prompter may or may not be the same person who originally created the issue

- In projects with a user base, users often submit issues (bug reports, feature requests)
 - A project maintainer then steps in, adds their perspective, and assigns the issue to Copilot
 - In this case, the maintainer who assigned Copilot is the prompter, not the original issue creator
- Copilot created the PR with the prompter as co-author
 - The prompter (now acting as PR manager) requested your review
 - Copilot may have also automatically reviewed the PR

As a non-manager reviewer, your role is to provide feedback, not to directly initiate more work by Copilot. The PR manager should remain in control of when and how Copilot makes additional changes.

Recommended review workflow:

1. Use “Comment” or “Request changes” based on the severity of issues:

- Use “**Comment**” for suggestions, questions, or minor issues that don’t block merging
- Use “**Request changes**” for significant issues that must be addressed before merging
- Both options allow you to provide feedback without directly triggering Copilot

2. Don’t ask Copilot to make changes directly:

- Avoid using features that would trigger Copilot to start working immediately
- Let the PR manager decide whether to ask Copilot to address your comments or make changes themselves

3. Write clear, actionable comments:

- Explain what needs to change and why
- Suggest specific solutions when appropriate
- The PR manager will decide how to address your feedback

For PR managers:

After receiving reviews from other team members:

1. Review all comments carefully:

- Decide which comments you agree with
- Dismiss or respond to comments you don’t entirely agree with
- This ensures Copilot only addresses feedback you’ve validated

2. Choose how to address valid feedback:

- **Option A:** Make the changes yourself (faster for simple fixes)

- **Option B:** Ask Copilot to address the feedback (better for complex changes)
- **Option C:** Add your own review summarizing which comments Copilot should address, then ask Copilot to respond to the open comment threads

3. Maintain clear communication:

- Let reviewers know how you plan to address their feedback
- Mark conversations as resolved after addressing them
- Request re-review from humans after Copilot makes significant changes
- Update the PR's "Assignees" field to reflect who is currently responsible for the PR

Transferring the PR manager role:

The original PR manager can hand over a PR to another person, who then becomes the new PR manager with control over Copilot's work on that PR. This might be useful when:

- The original PR manager is unavailable or on leave
- Someone with different expertise needs to guide the remaining work
- Responsibilities are being redistributed within the team

To transfer the PR manager role:

1. The original PR manager should clearly communicate the handover to all reviewers
2. The new PR manager should review the PR's history and any open feedback
3. The new PR manager should take over responding to Copilot and managing future iterations
4. The team should update the PR's "Assignees" field and comments or description to reflect the current PR manager

This workflow ensures the PR manager maintains control over the development process while benefiting from collaborative human review and Copilot's implementation capabilities.

Installing Claude Code on Windows

[Claude Code](#) is Anthropic's command-line coding agent. Installing it on Windows works well, but a few platform-specific pitfalls can cost you hours if you don't know about them. These notes capture a setup that works, and the gotchas to watch for.

Note

These notes were written in June 2026. As with the rest of this chapter, treat the specifics with caution— installers and behavior change quickly.

Run it from a Unix-like terminal

Claude Code expects a Unix-like shell. On Windows, run it from one of:

- **Git Bash** (ships with [Git for Windows](#));
- **MSYS2** (<https://www.msys2.org/>), which also gives you a package manager (**pacman**) and optional **zsh**;
- **WSL** (Windows Subsystem for Linux), the most Unix-faithful option.

If you use WSL, install Claude Code *inside* WSL— a Windows install will not carry over, because WSL has its own filesystem and **PATH**. Inside WSL the standard Linux install applies, and the Windows-specific **PATH** and **rehash** gotchas below don't apply.

Install Claude Code

There are two common routes:

1. **Native installer** (recommended):

```
curl -fsSL https://claude.ai/install.sh | bash
```

This installs the binary to `~/.local/bin`. If your organization's policy requires it, download and inspect the script before running it rather than piping it straight to **bash**.

2. **npm** (requires [Node.js](#)):

```
npm install -g @anthropic-ai/claude-code
```

! Claude Code migrates itself out of npm

Even if you install via npm, current versions **migrate themselves to a native install** the first time you run **claude** (at `~/.local/bin/claude`, or `~/.local/bin/claude.exe` on Windows) and remove the npm copy. You can watch this happen in the terminal output the first time you run **claude**; the current install methods are documented in the [Claude Code setup guide](#).

This is the single most confusing Windows gotcha: a path that worked a moment ago “disappears.” **Do not** hardcode the npm location (e.g. `.../AppData/Roaming/npm/...`) in your shell config. Point your **PATH** at the shell-appropriate directory shown in the next section instead.

Make sure your shell can find claude

The native binary lives in `~/.local/bin`. The installer adds this directory to **PATH** for ordinary shells, but on Windows two things commonly break that.

MSYS2 does not inherit the Windows PATH by default. It starts in a “minimal” path mode, so tools installed elsewhere on Windows are invisible to it. Add the binary’s directory explicitly in your `~/.zshrc` (or `~/.bashrc`). Because MSYS2’s `$HOME` is `/home/<you>` (its own home, not the Windows profile where the installer puts the binary), point `PATH` at the absolute Windows path:

```
export PATH="/c/Users/<you>/.local/bin:$PATH"      # MSYS2
```

In **Git Bash**, `$HOME` already is `/c/Users/<you>` (your Windows user profile), so the shorthand works there:

```
export PATH="$HOME/.local/bin:$PATH"      # Git Bash
```

Rehash after changing PATH. `zsh` and `bash` cache the locations of executables. If you add a directory to `PATH` in your shell config, the shell may still report **command not found** *even though the directory is on PATH*, because its command table is stale. Force a rebuild in the same shell right after editing `PATH`:

```
rehash      # zsh; use 'hash -r' in bash
```

This bites hardest with [oh-my-zsh](#), which builds `zsh`’s command table *before* your custom `PATH` line runs, so opening a fresh window may not clear the stale **command not found** on its own until you `rehash` (or move the `PATH` line before `oh-my-zsh` initializes).

Do not edit dotfiles with PowerShell redirection

PowerShell writes the wrong encoding

Never write to `~/.bashrc`, `~/.zshrc`, or other shell config files using PowerShell’s `>` or `>>` redirection. Windows PowerShell writes **UTF-16 with a byte-order mark**, which `bash/zsh` read as a stray character at the start of the file:

```
bash: $'\377\376export': command not found
```

Edit dotfiles from *inside* the shell (e.g. with `nano`, `vim`, or `echo 'export PATH=...' >> ~/.zshrc` run in `bash/zsh`), or with an editor that saves UTF-8 without a BOM (e.g. VS Code).

Authenticating the GitHub CLI in Git Bash or MSYS2

If you also use the [GitHub CLI](#) (`gh`) to push code or open pull requests, `gh auth login` may fail with:

```
could not prompt: ... running in MinTTY without pseudo terminal support
```

Git Bash and MSYS2 both default to the MinTTY terminal, which can't host the interactive prompt. Wrap the command with `winpty`, or run it from PowerShell / Windows Terminal instead:

```
winpty gh auth login
```

`winpty` ships with Git Bash, but in MSYS2 it's a separate package — install it first with `pacman -S winpty` if you get `command not found`.

Verify the install

Open a **new** terminal window (so it picks up your updated config) and run:

```
claude --version      # prints the installed version number
```

If you get a version number, you're ready to run `claude` in your project directory. If you get `command not found`, re-check the two `PATH` issues above: the directory must be on `PATH`, and you must `rehash` (or open a fresh window) after changing it.

Setting up MCP servers

The [Model Context Protocol](#) (MCP) is how a harness gains typed access to external systems. Configuring a server is usually a one-line command. Diagnosing one that *silently* isn't working is the part worth writing down, because the common failure mode produces no error at all — only a quiet absence of tools you assumed were there.

Note

These notes were written in July 2026, from a real diagnosis on a Linux machine. As with the rest of this chapter, treat the specifics with caution — harness internals and vendor defaults change quickly.

Installed is not registered

A server binary sitting on disk is not available to your agent. The harness knows only what its configuration declares, so installing `github-mcp-server` and *registering* it are two separate acts. Skipping the second is easy to miss, because the first one felt like the hard part.

The check is one command:

```
claude mcp list
```

A broken server can occupy the name your working one wanted

This is the failure worth knowing about, and it is nastier than a missing entry, because `claude mcp list` shows you something that looks right.

Plugin marketplaces can register servers of their own. An official GitHub plugin may install a **remote** server under exactly the name you meant to give your **local** one. The listing then reports a `github` server that is not your setup at all, and the local binary you installed goes unregistered and unnoticed.

So read the listing for the *transport and address*, not just the name:

```
plugin:github:github: https://api.githubcopilot.com/mcp/ (HTTP) - X Failed to  
connect - HTTP 400: ... Authorization header is badly formatted
```

An (HTTP) entry pointing at a vendor URL is a remote server. A local one shows a command path instead.

400 and 401 mean different things

The status code is the whole diagnosis here, and it is easy to skim past.

The configuration a plugin ships may hardcode a credential placeholder:

```
{"headers": {"Authorization": "Bearer ${GITHUB_PERSONAL_ACCESS_TOKEN}"}}
```

If that variable has no value when the harness starts, the header goes out as a bare **Bearer** with nothing after it. That is **malformed**, not **unauthorized**:

- **401** means the server read your token and rejected it — a real credential problem: expired, wrong scopes, wrong account.
- **400** means no token was ever substituted — a *configuration* problem, and no amount of re-issuing tokens will fix it.

Chasing a 400 as though it were a 401 sends you to the token-minting page for a problem that lives in a config file.

Install the binary the platform's own way

These notes started from a Linux install, where the binary lands under `$HOME/.local/bin`. On macOS the same server is a Homebrew formula, and getting it is one command:

```
brew install github-mcp-server
```

That installs to `/opt/homebrew/bin` on Apple silicon (`/usr/local/bin` under Intel Homebrew), so no manual download is needed.

The official install guide leads with a Docker recipe instead, and it has a catch worth knowing before you follow it: `docker` being on `PATH` does not mean the daemon is running. With Docker Desktop stopped, the server fails to connect to the Docker API, and that failure surfaces at *server start*, not at registration — so `claude mcp add` succeeds, and the break only shows up later, as a silent absence of tools. That is one more reason to prefer the binary path above.

Because the binary's location differs by platform and by installer, a launch wrapper should resolve it from `PATH` rather than hardcode it, with an override for the case where it isn't on one.

Supply credentials without storing a token

The obvious registration bakes a token straight into harness config:

```
claude mcp add github -e GITHUB_PERSONAL_ACCESS_TOKEN=<pat> -- <server> studio
```

That writes a live credential to a config file in plain text, and pins you to one token that will eventually expire.

A launch wrapper avoids both. It reads the credential at start time from a tool that already holds one, so nothing is stored and the server follows whatever account you are currently logged in as:

```
#!/bin/sh
set -eu

SERVER="${GITHUB_MCP_SERVER_BIN:-$(command -v github-mcp-server || true)}"
if [ -z "$SERVER" ]; then
    echo "github-mcp-server not on PATH; install it or set GITHUB_MCP_SERVER_BIN" >&2
    exit 1
fi

GITHUB_TOOLSETS="${GITHUB_TOOLSETS:-default,actions}"
```

```
export GITHUB_TOOLSETS

GITHUB_PERSONAL_ACCESS_TOKEN="$(gh auth token)"
if [ -z "$GITHUB_PERSONAL_ACCESS_TOKEN" ]; then
    echo "empty token; run 'gh auth login'" >&2
    exit 1
fi
export GITHUB_PERSONAL_ACCESS_TOKEN

exec "$SERVER" stdio "$@"
```

Register the wrapper rather than the binary:

```
claude mcp add --scope user github -- ~/.local/bin/github-mcp-server-stdio
```

Note the explicit failure when the token comes back empty. A wrapper that silently exports an empty string reproduces the bare-Bearer bug you just finished diagnosing.

Toolsets are opt-in, and the default may omit what you need

A server does not necessarily expose everything it can do. GitHub’s server exposes a default group, and that default carries **no continuous-integration access at all** — no workflow runs, no job logs, no re-run trigger. As of this writing (server v1.7.0) the actions toolset’s four tools are `actions_get`, `actions_list`, `actions_run_trigger`, and `get_job_logs` — the exact names have moved around across releases, so treat this list as a snapshot rather than a promise.

If your workflow involves driving pull requests to a clean state, that omission matters, because reading check status is most of the job. Request the extra group explicitly:

```
GITHUB_TOOLSETS=default,actions
```

Note that the selection **replaces** the default rather than extending it, which is why the value above names `default` explicitly. Writing `actions` alone would silently trade away every default tool for the four you asked for — a net *loss* that looks like a successful configuration change.

So compare the tool list before and after, and confirm the count went up rather than sideways. On the same v1.7.0 server, the default group carries 44 tools, and `default,actions,discussions,dependabot,labels,notifications` carries 63 — the count moving the right direction, as the check above expects.

subscribe_pr_activity isn't a local-server tool

Workflow guidance written for remote or web agent sessions sometimes names `subscribe_pr_activity` as the way to watch a pull request's activity. It doesn't appear in a locally-run GitHub MCP server, under any toolset combination.

The local analogues are `manage_notification_subscription` and `manage_repository_notification_subscription`. Reach for those instead when working from a local harness.

Verify with a real call, and expect to restart

Two habits close this out.

Verify by calling, not by reading a list. A tool appearing in the registry proves the harness parsed a config file. It does not prove the server started, authenticated, or can reach the API. One cheap identity call plus one read (for GitHub: `get_me`, then listing pull requests on a repo you know) proves the whole path end to end.

Expect the tools to be missing until you restart. MCP servers connect when a session starts, so a server registered mid-session is inert for the rest of it. This is a common false alarm: the registration worked, and the tools genuinely are not there yet.

Finally, note which new tools can *write*. A re-run or dispatch tool can trigger CI, and permissive permission modes will not prompt before it does. Treat those the way you would treat a merge — something a human authorizes, not something an agent does in passing.

Managing Gemini API Spend and Cost Optimization

This guide describes how to manage Google AI Studio and Google Cloud Gemini API spend caps, unpause paused API services, and optimize token consumption across local tools and GitHub Actions workflows.

Identifying Paused Projects (Project Numbers vs. Friendly Names)

When a project reaches its monthly budget limit, Google sends an email notification stating that Gemini API service has been paused. These email notifications identify the affected project using its internal **numerical GCP Project Number** (e.g. 156839315029).

In contrast, [Google AI Studio](#) lists projects by their **friendly display names** (such as `ai-config Project` or `gha-project`) and alphanumeric client IDs (`gen-lang-client-...`).

To find which project in AI Studio corresponds to the notification email:

1. Open [Google AI Studio Projects](#).
2. Locate the project corresponding to your client ID or spend alert.

3. Any project that has hit its monthly spend cap will have its API requests paused until the limit is updated.

Adjusting and Unpausing Spend Caps

To restore API access for a paused project:

1. Open [Google AI Studio Spend](#).
2. Increase the monthly spend cap dollar amount or set it to unlimited.
3. Save your changes. API requests will automatically resume within a few minutes.

If no manual action is taken, accumulated spend resets to **\$0 on the 1st of the next month**, and API service automatically resumes up to the configured cap.

Setting Up Google Cloud Budget Alerts

To receive early warnings before reaching a spend cap:

1. Open [Google Cloud Console Billing Budgets & Alerts](#).
2. Select your billing account and click **Create Budget**.
3. Name the budget (e.g., Lab AI API Monthly Budget) and select the relevant GCP projects.
4. Set your target monthly budget amount.
5. Configure trigger rules for email notifications at **50%**, **75%**, and **90%** of the budget.
6. Click **Finish**. You will receive email alerts before any project hits its spend cap.

Cost Optimization Best Practices

To maximize the efficiency of your API spend across local CLI sessions, subagents, and automated workflows:

- **Right-Size Model Selection:** For general Gemini API, Python SDK, or custom scripts, prefer Flash-tier models (such as `gemini-2.5-flash`) over Pro-tier models (`gemini-2.5-pro`). Flash models provide a substantially lower token cost for routine search, log parsing, and background processing. For Antigravity Agent workflows (`google-antigravity`), the agent defaults to `gemini-3.7-flash` (already a Flash-tier model). Supported Antigravity Agent model options include `gemini-3.7-flash`, `gemini-3.6-flash`, `gemini-3.5-flash`, and `gemini-3.5-flash-lite`.
- **Use Context Caching:** For workloads that repeatedly pass static context over 2,048 tokens (such as large reference docs, system prompts, or codebase indices), use Gemini Context Caching. Cached input tokens receive a substantial discount compared to standard input tokens.

- **Use the Batch API for Non-Realtime Tasks:** For offline batch processing, evaluation suites, or background doc updates, submit requests via the Gemini Batch API to receive a 50% discount on input and output tokens.
- **GitHub UI Diff Collapsing:** Mark dependency lockfiles (`*.lock`, `package-lock.json`, `yarn.lock`, `renv.lock`) and generated build artifacts as `linguist-generated=true` in `.gitattributes` to collapse them in GitHub’s web diff view and exclude them from repository language statistics.

References

- 2001: A Space Odyssey*. 1968. Film. [https://en.wikipedia.org/wiki/2001:_A_Space_Odyssey_\(film\)](https://en.wikipedia.org/wiki/2001:_A_Space_Odyssey_(film)).
- Asimov, Isaac. 1950. *I, Robot*. Novel; Gnome Press. https://search.library.ucdavis.edu/permalink/01UCD_INST/9fle3i/alma990000226350403126.
- Battlestar Galactica*. 2004. Television Series. [https://en.wikipedia.org/wiki/Battlestar_Galactica_\(2004_TV_series\)](https://en.wikipedia.org/wiki/Battlestar_Galactica_(2004_TV_series)).
- Belcak, Peter, Greg Heinrich, Shizhe Diao, et al. 2025. *Small Language Models Are the Future of Agentic AI*. NVIDIA Research; arXiv preprint. <https://arxiv.org/abs/2506.02153>.
- Blade Runner*. 1982. Film. https://en.wikipedia.org/wiki/Blade_Runner.
- Card, Orson Scott. 1985. *Ender’s Game*. Novel; Tor Books. https://en.wikipedia.org/wiki/Ender%27s_Game.
- Dettmers, Tim, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. *QLoRA: Efficient Finetuning of Quantized LLMs*. arXiv preprint. <https://arxiv.org/abs/2305.14314>.
- Flight of the Conchords. 2007. *The Humans Are Dead*. Music Video. <https://www.youtube.com/watch?v=B1BdQcJ2ZYY>.
- Herbert, Frank. 1965. *Dune*. Novel; Chilton Books. https://en.wikipedia.org/wiki/Organizations_of_the_Dune_universe#Thinking_machines.
- Hu, Edward J., Yelong Shen, Phillip Wallis, et al. 2021. *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv preprint. <https://arxiv.org/abs/2106.09685>.
- LeCun, Yann. 2022. *A Path Towards Autonomous Machine Intelligence*. Meta AI Research; New York University; Technical Report. <https://openreview.net/forum?id=BZ5a1r-kVsf>.

Terminator 3: Rise of the Machines. 2003. Film. https://en.wikipedia.org/wiki/Terminator_3:_Rise_of_the_Machines.

The Matrix. 1999. Film. https://en.wikipedia.org/wiki/The_Matrix.

WarGames. 1983. Film. <https://en.wikipedia.org/wiki/WarGames>.